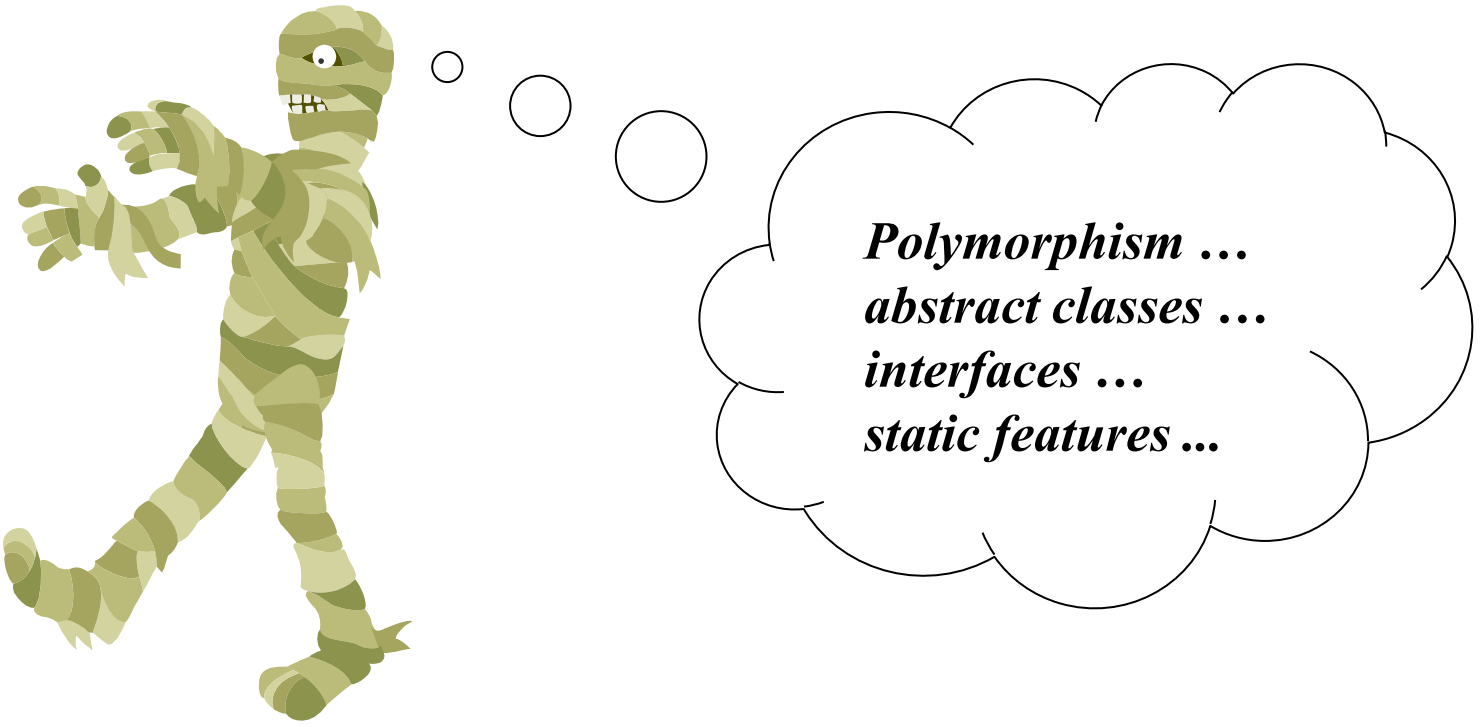
Chapter 6 Advanced Class Design



Contents



- Polymorphism
- Abstract Classes
- Interfaces
- Static Features
- Design Pattern
- Singleton Pattern
- Strategy Pattern

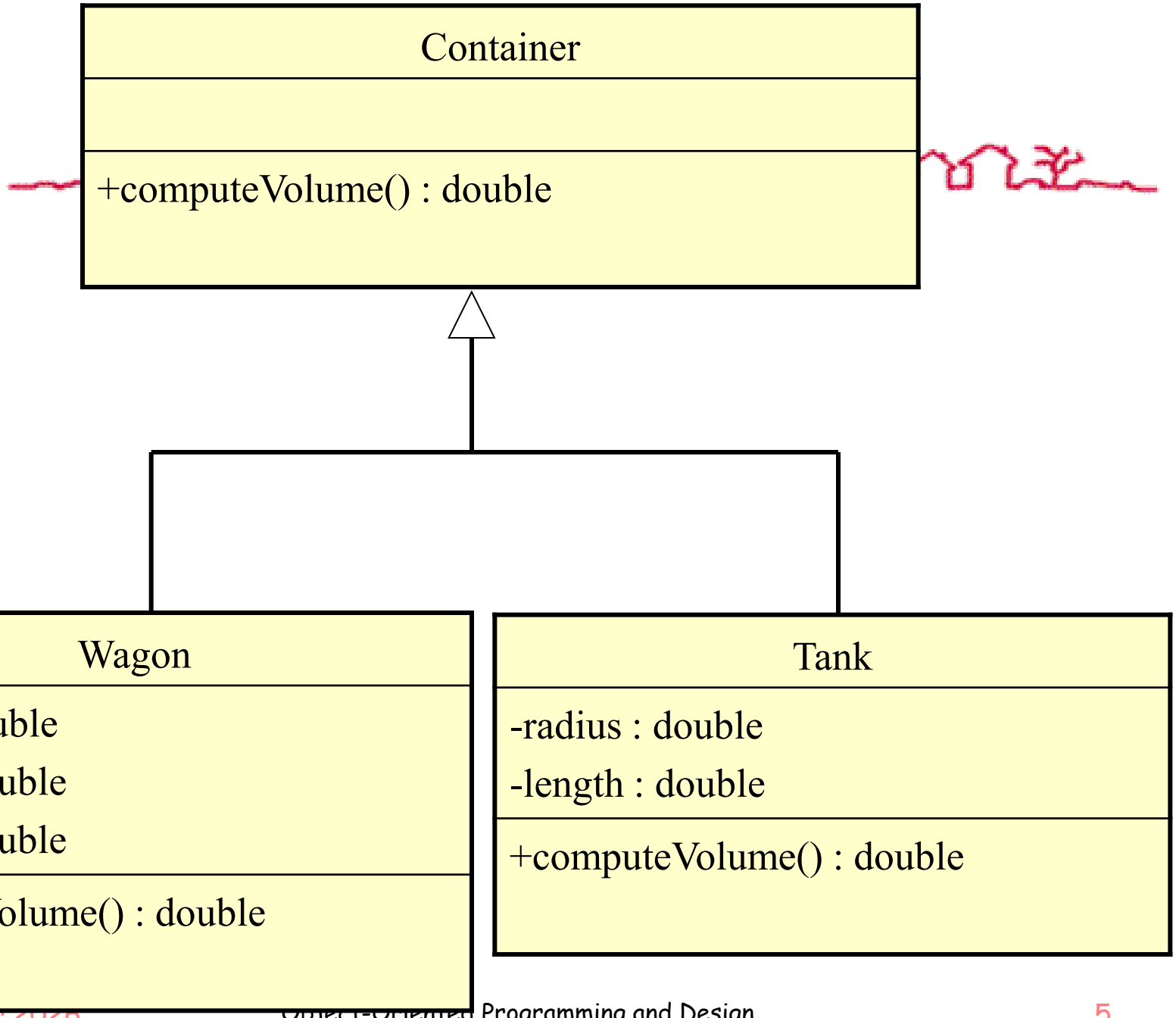
6.1 Polymorphism



Polymorphism

- The ability of two or more objects belonging to different classes to respond to exactly the same message in different class-specific ways
  - A surgeon, a hair stylist, and an actor

Eg:


Container
+computeVolume() : double
Wagon
-width : double
-height : double
-length : double
+computeVolume() : double
Tank
-radius : double
-length : double
+computeVolume() : double

Class Container

```
public class Container {

  /**
   * Computes the volume of this container.
   *
   * @return the volume of this container.
   */
  public double  computeVolume(){

     System.out.println("This is in Container.");
     return 0.0;
  }
}
```

```
public class  Wagon  extends  Container  {
  private double  width;
  private double  height;
  private double  length;

  public Wagon(double initialWidth,
                        double initialHeight,
                        double initialLength)  {
      width = initialWidth;
      height = initialHeight;
      length = initialLength;
  }

  public double  computeVolume()  {
      return width * height * length;
  }
}
```

```
public class  Tank  extends  Container  {
  private double  radius;
  private double  length;

  public Tank(double initialRadius,
                      double initialLength) {
      radius = initialRadius;
      length = initialLength;
   }

  public double  computeVolume() {
      return Math.PI * radius * radius * length;
    }
}
```

```
import  java.io.*;
public class ContainerDemo {
  private static PrintWriter  stdOut =          new
  PrintWriter(System.out, true);
  public static void  main(String[]  args) {
      Container [ ]container = new Container[5];
      container[0] = new Wagon(1, 1, 3);
      container[1] = new Tank(1, 3);
      container[2] = new Wagon(2, 3, 4);
      container[3] = new Tank(2, 3);
      container[4] = new Container();
      for(int i = 0; i < 5; i++){
            stdOut.println("Volume: " +
  container[i].computeVolume());
      }
  }
}
```

Output

```
Volume: 3.0
Volume: 9.42477796076938
Volume: 24.0
Volume: 37.69911184307752
This is in Container.
Volume: 0.0
```

Polymorphism, cont.

- The term **polymorphism** is defined in Webster's dictionary as:
  - *'The quality or state of being able to assume different forms'.*
- The line of code:

```
container.computeVolume()
```

is said to be **polymorphic** because the method code performed in response to the message can take many different forms, depending on the class identity of the object.

Polymorphism, cont.



- Polymorphism
  - Inheritance
  - Overriding
  - Superclass object reference
- Polymorphism doesn't happen automatically in C++
  - Virtual functions

Polymorphism Simplifies Code Maintenance

- To appreciate the power of polymorphism, let's look at how we might have to approach the challenge of handling different objects in different type-specific ways with a non-OO programming language
  - We'd typically handle all of the various scenarios having to do with different kinds of containers using a series of 'if' tests (see next page)

Simplified Maintenance, cont.

```
for (int i = 0; i < 5; i++) {
    // Process the ith container.
    Container c = (Container) container.elementAt(i);
    // (pseudocode)
    if (c is a wagon)
      computeWagonVolume(c);
    else if (c is a tank)
      computeTankVolume(c);
    else if ...
}
```

- As the number of cases grows, so too does the 'spaghetti' nature of the resultant code!

Simplified Maintenance, cont.



- With a polymorphic language, we can invent as many subtypes after the fact as we like and the client code that manipulates these needn't change!

```
// SAME CODE AS BEFORE!
// Step through the array (collection) ...
for(int i = 0; i < 5; i++){
        stdOut.println("Volume: " +
container[i].computeVolume());
}
```

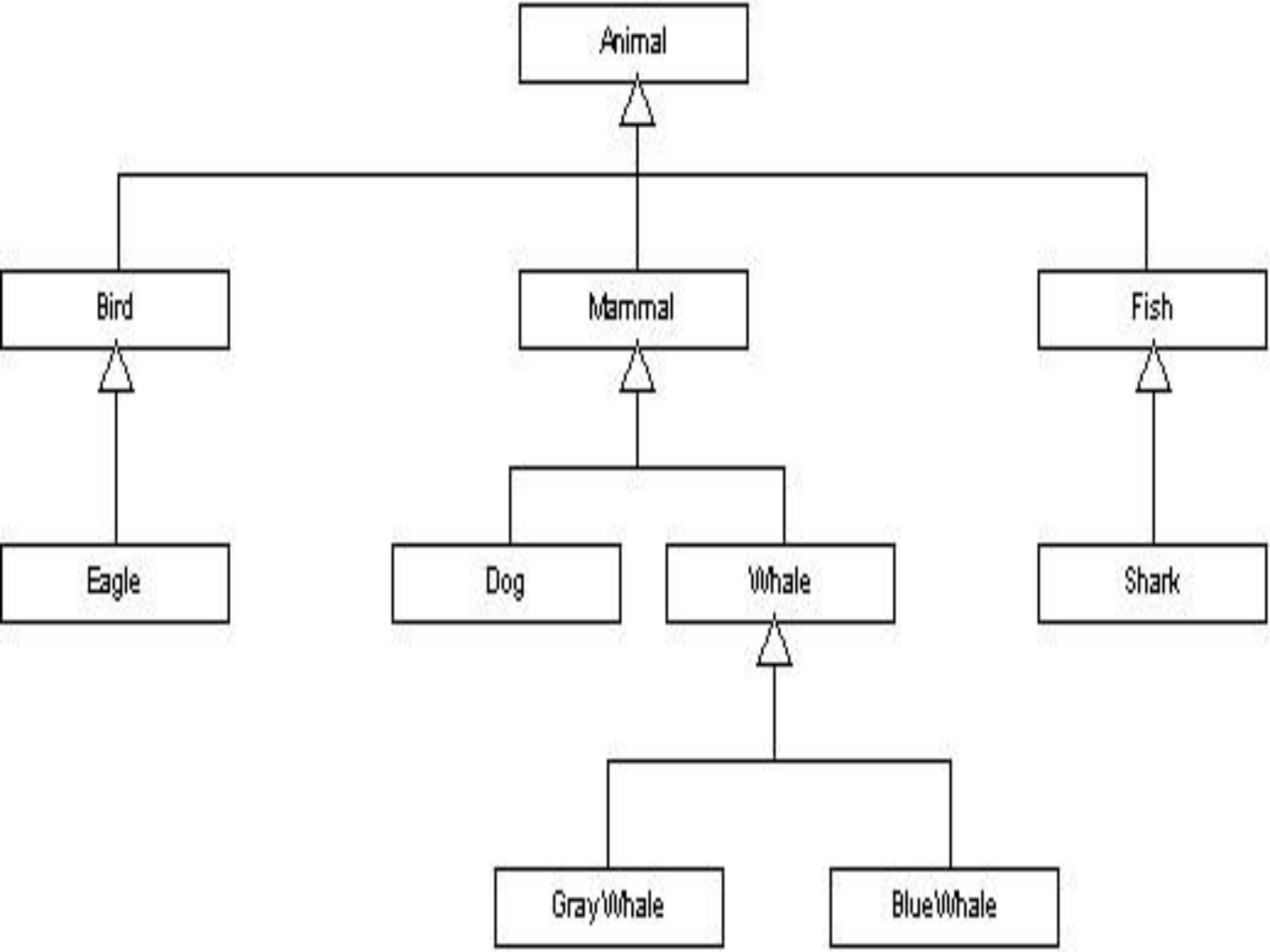
6.2 Abstract Class



Find relationships between the classes



- Fish
- BlueWhale
- Dog
- Animal
- Eagle
- Shark
- Mammal
- GrayWhale
- Bird
- Whale


Animal
Bird
Mammal
Fish
Eagle
Dog
Whale
Shark
GrayWhale
BlueWhale

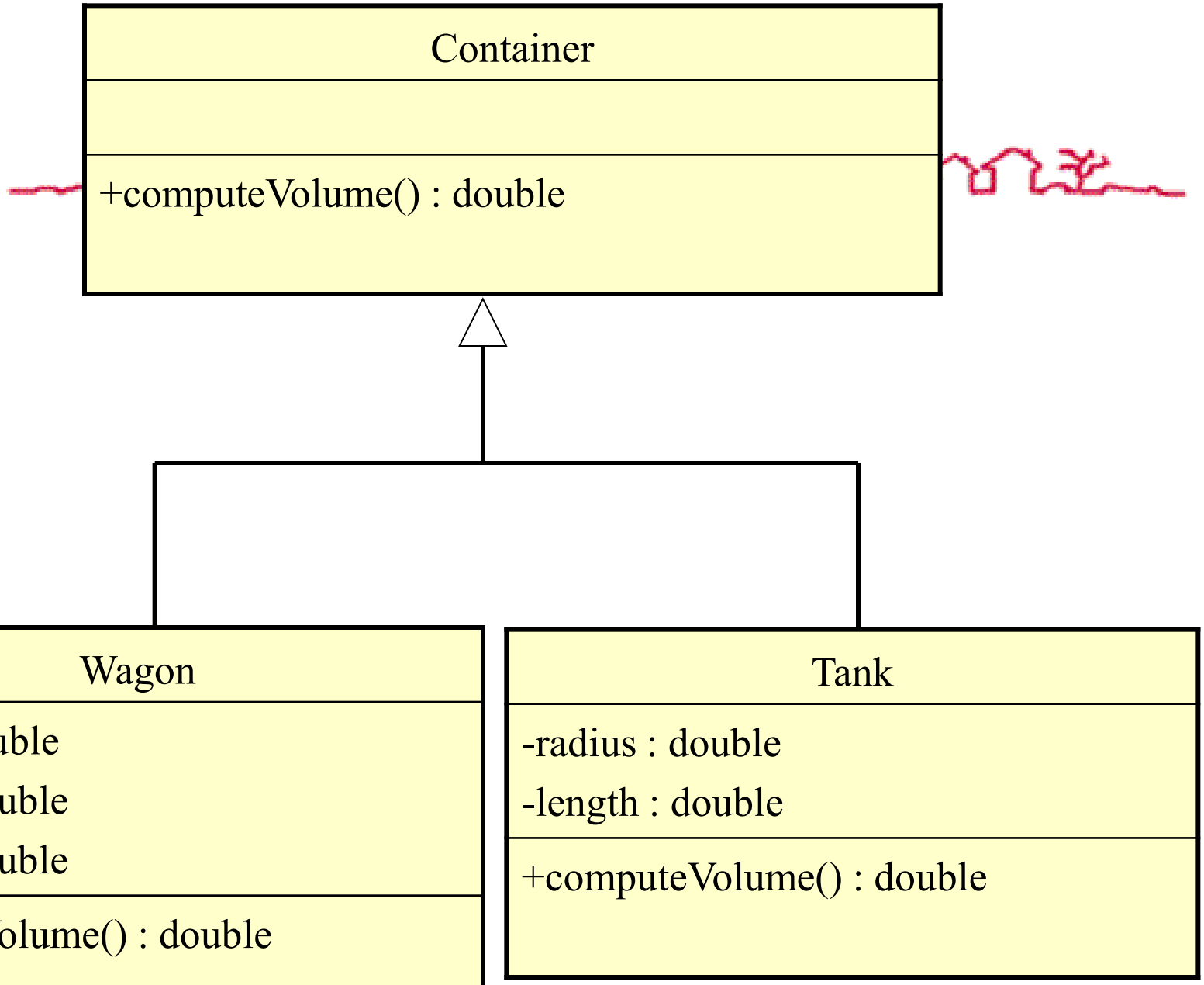
Abstract Class



- Abstract class
  - Can not be instantiated.
  - Use **`abstract`** keyword.
- Definition
  - **`public abstract class`** ***`className`***

```
{ . . . }
```

Eg:


Container
+computeVolume() : double
Wagon
-width : double
-height : double
-length : double
+computeVolume() : double
Tank
-radius : double
-length : double
+computeVolume() : double

Container Class

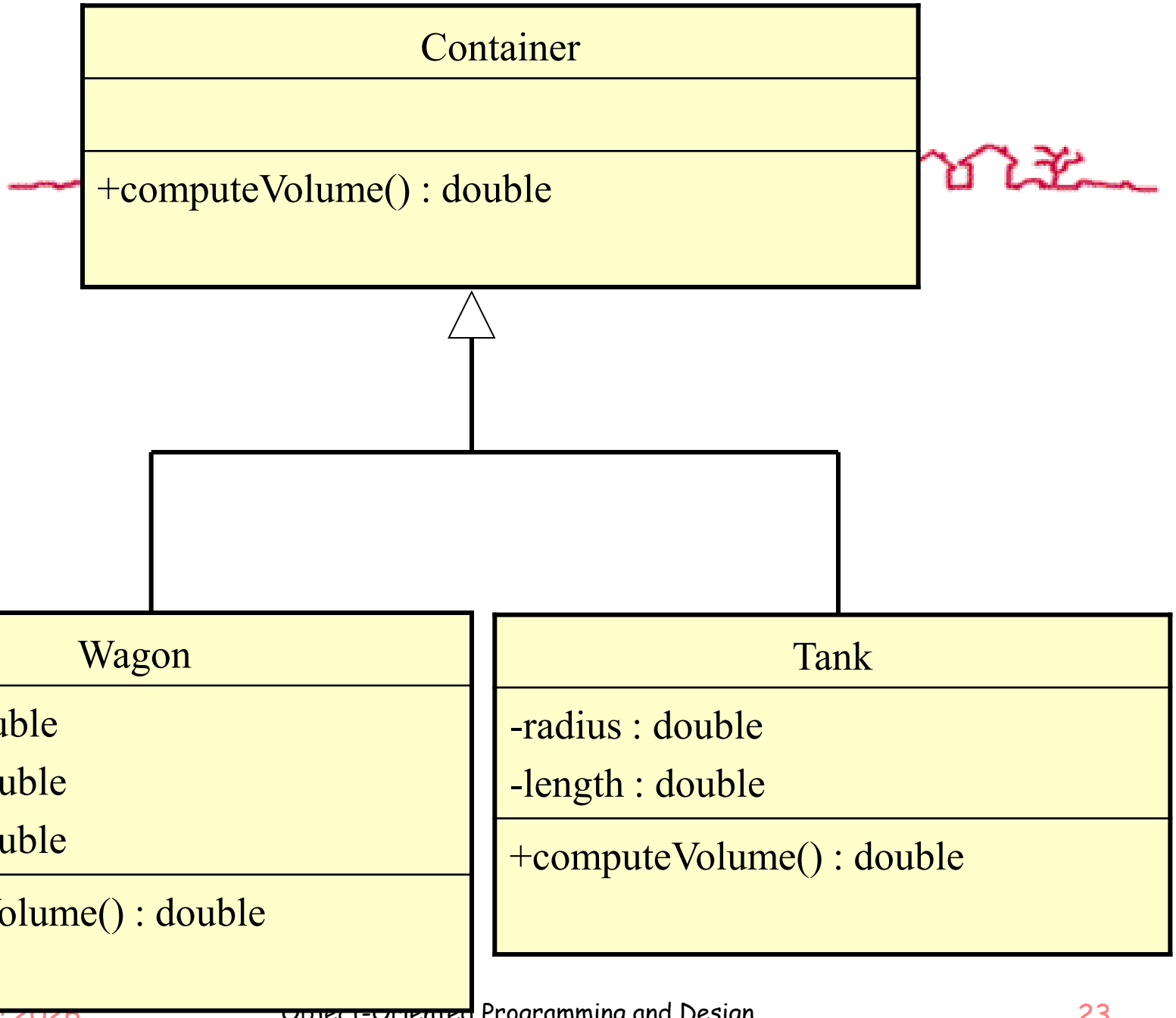
```
public abstract class Container {
/**
 * Computes the volume of this container.
  * @return the volume of this container.
 */
     public double computeVolume(){
          return 0.0;
     }
}
```

Abstract Methods



- Abstract Method's definition:
  - No method bodies;
  - Use **abstract** keyword;
- Abstract methods's declaration:
  - **public abstract void eat(int amount);**
  - **public abstract void sleep(int hours);**
- Abstract methods must be defined in abstract class.

Eg:


Container
+computeVolume() : double
Wagon
-width : double
-height : double
-length : double
+computeVolume() : double
Tank
-radius : double
-length : double
+computeVolume() : double

Container Class

```
public abstract class Container {
/**
 * Computes the volume of this container.
  * @return the volume of this container.
 */
    public abstract double computeVolume();
}
```

Abstract Classes and Inheritance, cont.

- If we fail to provide concrete method bodies for all of the abstract methods that we've inherited:
  - They are inherited as abstract methods.
  - The compiler will force us to declare the subclass as an abstract class.

```
public abstract class Wagon  extends  Container  {
  private double  width;
  private double  height;
  private double  length;

  public Wagon(double initialWidth,
                        double initialHeight,
                        double initialLength) {
      width = initialWidth;
      height = initialHeight;
      length = initialLength;
  }

  public abstract double  computeVolume();
}
```

```
public class  Tank  extends  Container  {
  private double  radius;
  private double  length;

  public Tank(double initialRadius,
                      double initialLength) {
      radius = initialRadius;
      length = initialLength;
   }

  public double  computeVolume() {
      return Math.PI * radius * radius * length;
    }
}
```

Abstract Classes and Instantiation



- We cannot instantiate an abstract class:

```
Container c = new Container();  // Impossible!
```

- This makes intuitive sense, because there are service(s) that such an object could not perform

```
c.computeVolume(); // Behavior undefined!
```

Abstract Classes and Variable Declaration



- We are allowed to declare references belonging to an abstract class, however

```
Container c;  // Fine and dandy!
```

- This enables us to take advantage of polymorphism, e.g., when iterating through a collection

```
Vector container;
for (int i = 0; i < container.size(); i++) {
Container c =
  (Container)container.elementAt(i);
  c.computeVolume() // polymorphic
}
```

Abstractness



- Recall that a class, as an abstract data type, is an abstraction of a real world object
- We can see that an abstract class is ‘more abstract’ than a ‘normal’ class because we’ve omitted the details for how one or more particular behaviors are to be performed
- But, with an abstract class, we typically still specify the data structure (attributes)
- What if we wanted to specify *only* abstract behaviors?

6.3 Interface



Interfaces

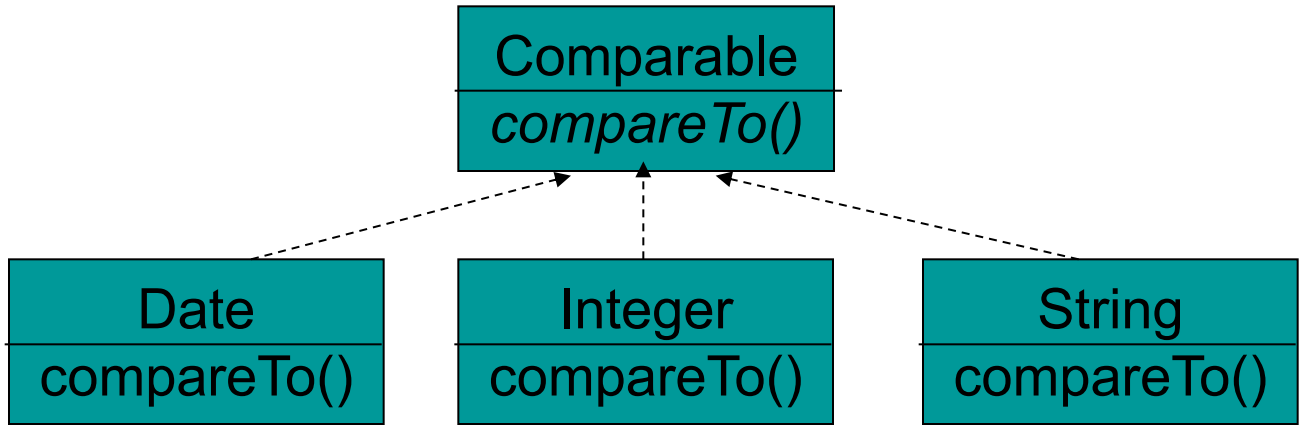


- An interface is simply a named list of method declarations
  - Does not implement methods, only declares them
  - An interface is not a class, although it looks remarkably like an abstract class
- Declared types in Java are either classes or interfaces
- Interfaces represent a promise of support services to the objects which implement the interface – a “contract”

```
public interface File {
    public void open(String name);
    public void close();
}
```

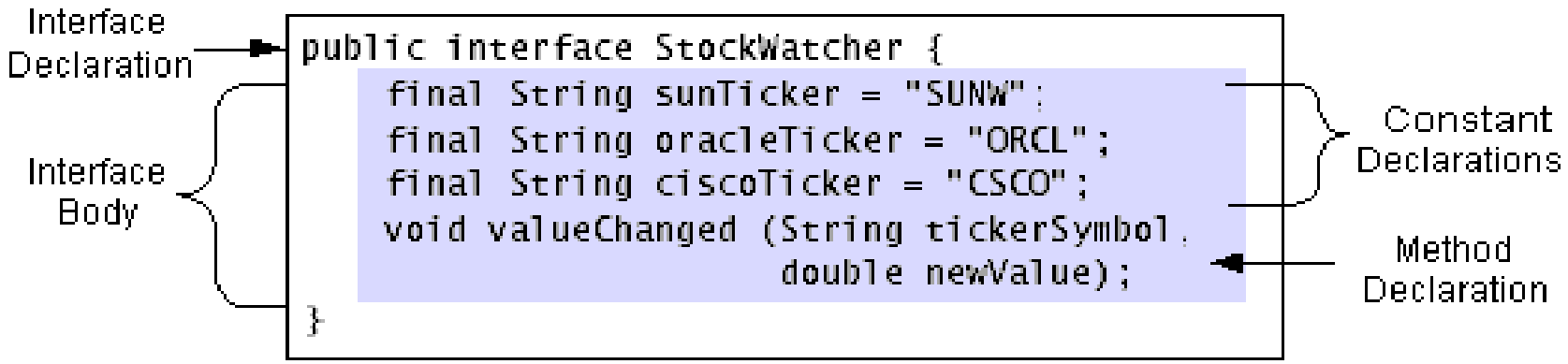
Protocols

- An interface defines a protocol (a set of methods)
  - If a class implements a given interface, then that class implements that protocol
- An interface can impose a common protocol on a group of classes that are not related by inheritance
  - In the chain of classes related by inheritance, the common protocol is imposed through subclassing



Declaring an Interface

- Interface definition has two components
  - Interface declaration
  - Interface body
- The interface declaration declares various attributes about the interface
  - Name
  - Whether it extends other interfaces



Implementing Interface Methods



- Methods declared in an interface are implemented in the classes which support that interface

```
public class TextFile implements File {
    public void open(String name) {
        // implementation of open method
    }
    public void close() {
        // implementation of close method
    }
}
```

Syntax



- In a class declaration, the naming of the superclass precedes any interfaces supported by the class:

```
public class Directory extends Secure
             implements File {
     ...
}
```

Multiple Interfaces



- If a class implements multiple interfaces, the interfaces are all listed, separated by commas:

```
public class Directory
             implements File, Secure {
    ...
}
```

Implementing an Interface



- A class which implements an interface *must* implement *every* method defined by that interface
- If one or more methods is not implemented, Java will generate a compiler error
- Subclasses automatically implement all interfaces that their superclass implements

Cloneable Interface



- Some interfaces, such as **Cloneable**, do not contain any methods
  - In the case of **Cloneable**, it alerts the JVM that objects of the implementing class can be copied via the `clone()` method
  - These act as a signal to Java that a class observes a certain protocol that may not be expressible as a specific set of methods
  - Only the objects that implement the Cloneable interface can be copied, or cloned.

Typing and Interfaces



- A variable's type can be an interface
- Stipulations
  - Only objects whose class implements that interface can be bound to that variable
  - Only messages defined by the interface can be used
  - Interfaces cannot appear in a `new` expression

```
File r = new File(); // Error
File f = new TextFile(); // OK!
```

Subinterfaces



- Interfaces can be extended
  - Interface hierarchy is independent of the class hierarchy
  - The interface which extends another interface inherits all of its method declarations

```
interface File {
   public void open(String name);
   public void close();
}
interface ReadableFile extends File {
   public byte readByte();
}
interface WritableFile extends File {
   public void writeByte(byte b);
}
interface ReadWriteFile extends ReadableFile,
                     WritableFile {
   public void seek(int position);
}
```

Using Interfaces



- Using interfaces allows
  - Cross-hierarchy polymorphism
  - Access to methods in separate class trees
  - Substitution of an object for another object which is not related via the class hierarchy
- Classes that implement the same interface understand the same messages
  - Regardless of their location in the class hierarchy

More Advantages of Using Interfaces



- Allows more control over how objects are used
- The programmer can define a method's parameters as interfaces
  - This restricts the use of those parameters
  - The programmer knows which message the objects will respond to
- Improves reusability of code

Naming Conventions for Interfaces



- Make "able" interfaces
  - Cloneable, Serializable, ...
- Name interfaces using proper nouns and provide "Impl" implementation of your interfaces
  - Bank, BankImpl, BankAccount, BankAccountImpl
  - With this convention, the interface typically contains a definition for all (or most) of the implementation class' public methods
- Prefix interface names with "I" and use proper nouns for your classes
  - IBank, Bank, IBankAccount

Interfaces



- The preferred approach is to declare a Java **interface**, which is a set of (abstract) method signatures that collectively define what it means to assume a certain role (such as teaching)

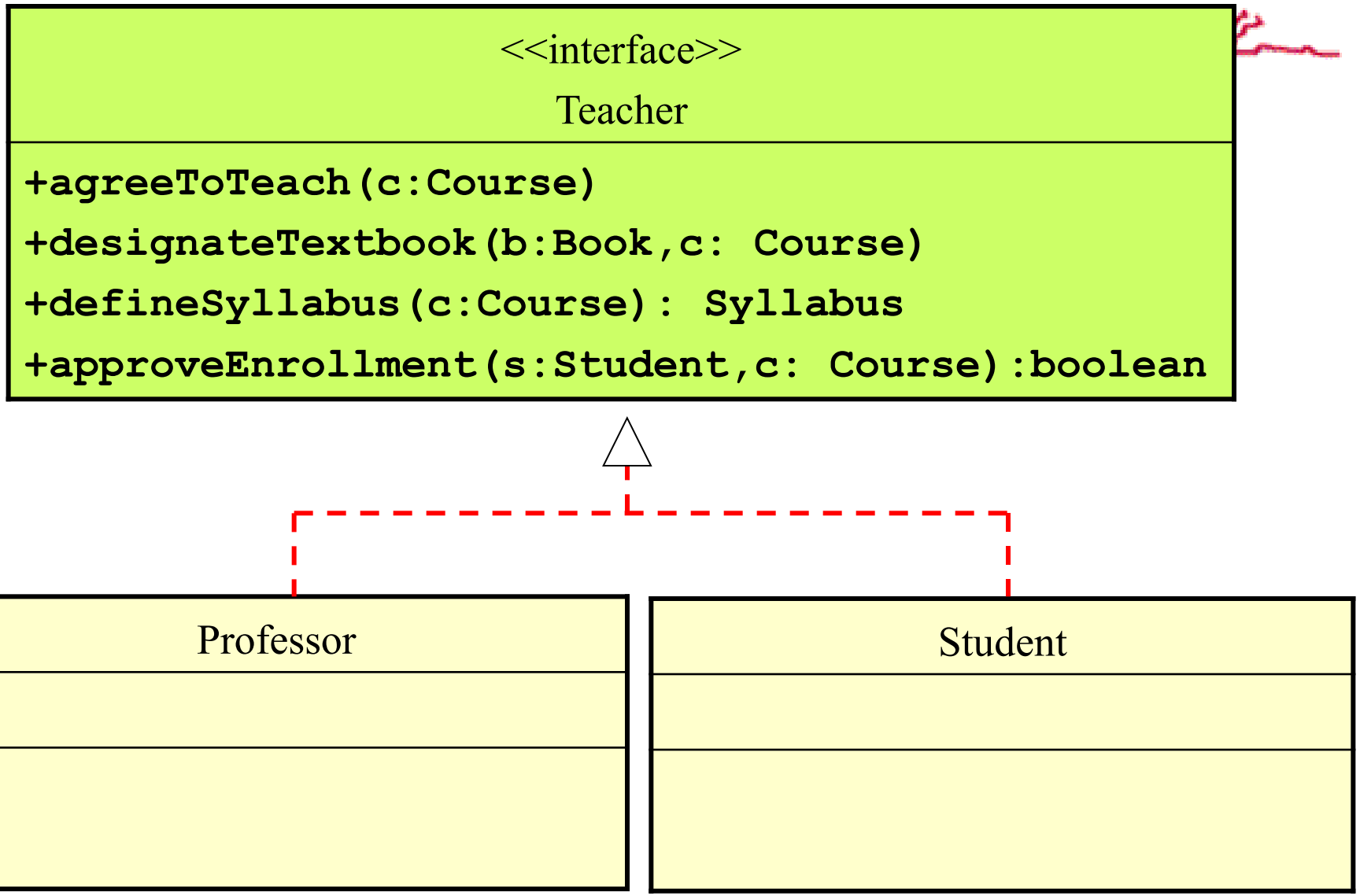
```
public interface Teacher {
  // Note:  no "abstract" keyword required.
  public void agreeToTeach(Course c);
  public void designateTextbook(Book b, Course c);
  public Syllabus defineSyllabus(Course c);
  public boolean approveEnrollment(Student s,
  Course c);
}
```

Interfaces, cont.

- We then indicate that a particular class performs these behaviors by implementing the interface and overriding the methods (just as we did with abstract classes):

```
public class Professor implements Teacher {
    public void agreeToTeach(Course c) {
        code body provided …
    }
    public void designateTextbook(Book b,
Course c) {
        code body provided …
    }
}
```

UML Diagram


<<interface>>
Teacher
+agreeToTeach(c:Course)
+designateTextbook(b:Book,c: Course)
+defineSyllabus(c:Course): Syllabus
+approveEnrollment(s:Student,c: Course):boolean
Professor
Student

Interfaces, cont.



- In terms of the ‘abstractness spectrum’, an interface is even more abstract than an abstract class, which is in turn more abstract than a ‘regular’ class
- A class may be instructed to implement as many interfaces as desired (by contrast, a class may extend only one parent class)
- For example, if we were to invent a second interface called ‘Administrator’, which specified the following method signatures:

```
public boolean approveNewCourse(Course c);
public boolean hireProfessor(Professor p);
```

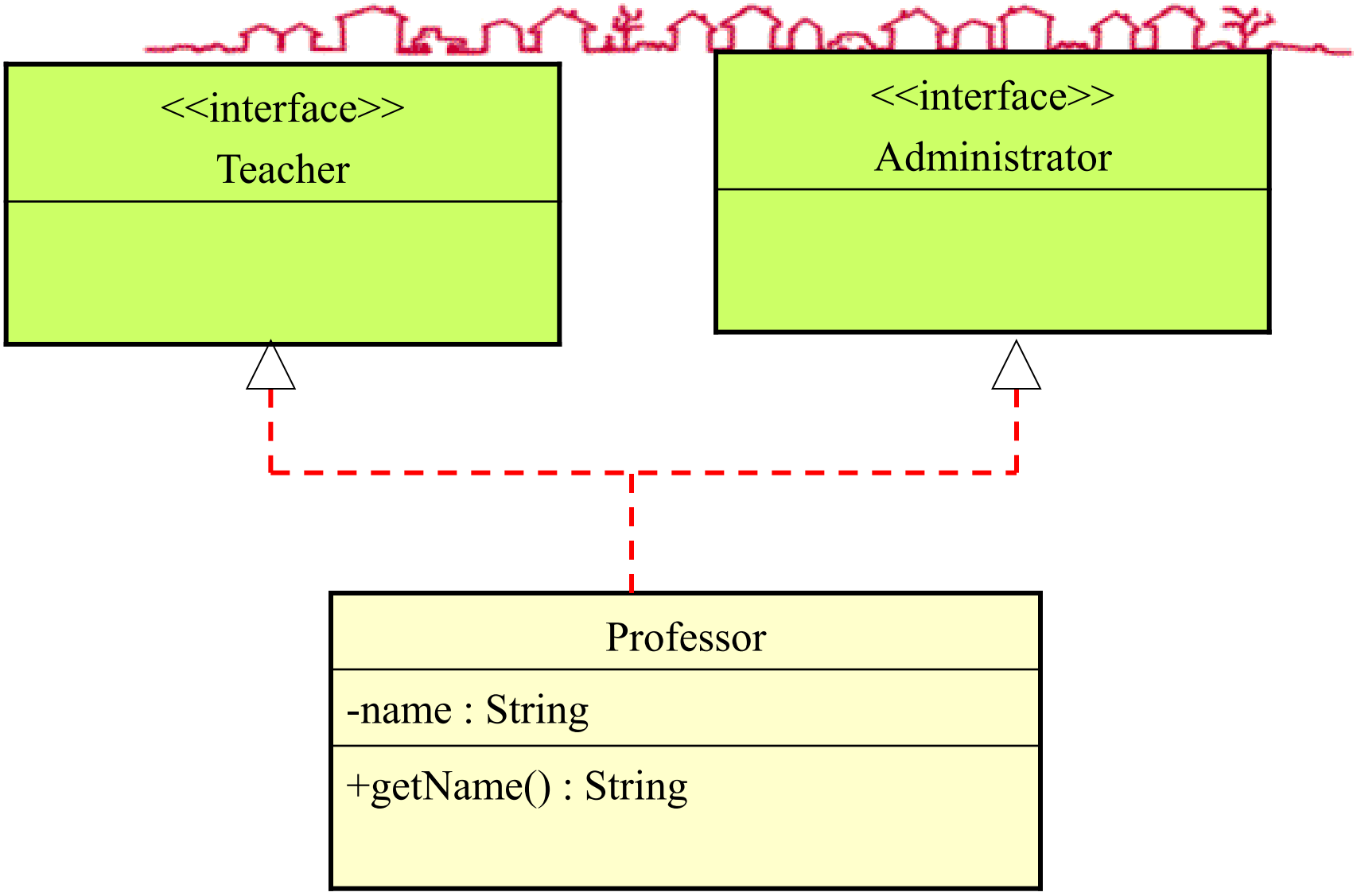
Interfaces, cont.

- then we could instruct a class to implement both the Teacher and Administrator interfaces:

```
class Professor implements Teacher,
Administrator { ... }
```

- When a class implements more than one interface, it effectively assumes multiple identies or roles
  - Its 'handle' can therefore be managed by various types of reference variables

UML Diagram


<<interface>>
Teacher
<<interface>>
Administrator
Professor
-name : String
+getName() : String

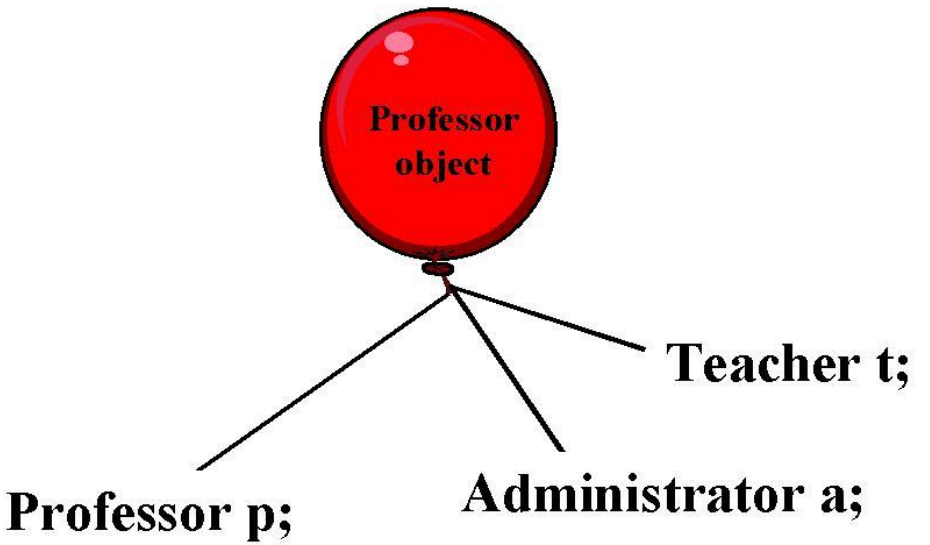
Interfaces, cont.



```
Professor p = new Professor();
Teacher t;
Administrator a;

t = p;
a = p;
```

Interfaces, cont.



A Professor object can be "handled' by Professor, Teacher, and Administrator references, because a Professor is a Teacher, and a Professor is an Administrator!

- An interface is therefore another way of implementing the "is a" relationship!

Interfaces, cont.

- We may then command the same object as either a Professor …

```
p.getName();
```

or as a Teacher …

```
t.agreeToTeach(c);
```

or as an Administrator …

```
a.approveNewCourse(c);
```

because it is all three!

Interfaces, cont.

- We cannot do the following, however:

```
t.getName();
```

- because even though, in our scenario, t is referring to a Professor object at runtime, the compiler will object: not all Teachers are Professors

Interfaces, cont.



- If we also wish to enable Students to be teachers:

```
class Professor implements Teacher,
Administrator { … }
class Student implements Teacher { … }
```

- we may then use Student and Professor object references interchangeably wherever a Teacher object reference is required

Versatility of Interfaces



```
public class Course {
    Teacher instructor;  // This is a more versatile
design.
    public void setInstructor(Teacher t) {
          instructor = t;
    }              // etc.
```

```
public static void main(String[] args) {
    Student s = new Student();
    Professor p = new Professor();
    Course c = new Course();
    c.setInstructor(p);   -or-    c.setInstructor(s);
```

Either will work just fine!

Interfaces, cont.

- An interesting example of casting to consider:

```
Professor p = new Professor();
Student s = new Student();

Teacher t;
t = p;
p = t;
```

Interfaces, cont.

- An interesting example of casting to consider:

```
Professor p = new Professor();
Student s = new Student();
Teacher t;

t = p;  // This is fine.
p = (Professor) t; // Will compile and run fine now.
}
}
```

Interfaces, cont.

- An interesting example of casting to consider:

```
// implements Teacher
Professor p = new Professor();
// implements Teacher
Student s = new Student();
Teacher t;
t = p;  // This is fine.
//Will compile and will run fine.
p = (Professor) t;
// t now refers to our Student object.
t = s;
// Will this compile/run properly?
p = (Professor) t;
```

Interfaces, cont.

- An interesting example of casting to consider:

```
Professor p = new Professor();
Student s = new Student();
Teacher t;
t = p;
p = (Professor) t;
t = s;
p = (Professor) t;
// Will compile, but will throw a runtime ClassCastException.
}}
```

Interfaces vs. Multiple Inheritance, cont.



- There are object modeling techniques for making up for the lack of multiple inheritance in a language
  - See Object-Oriented Modeling and Design by James Rumbaugh et al, Prentice Hall, 1991; Chapter 4, Section 4.4 "MULTIPLE INHERITANCE", section 4.4.3 "Workarounds"

Versatility of Interfaces



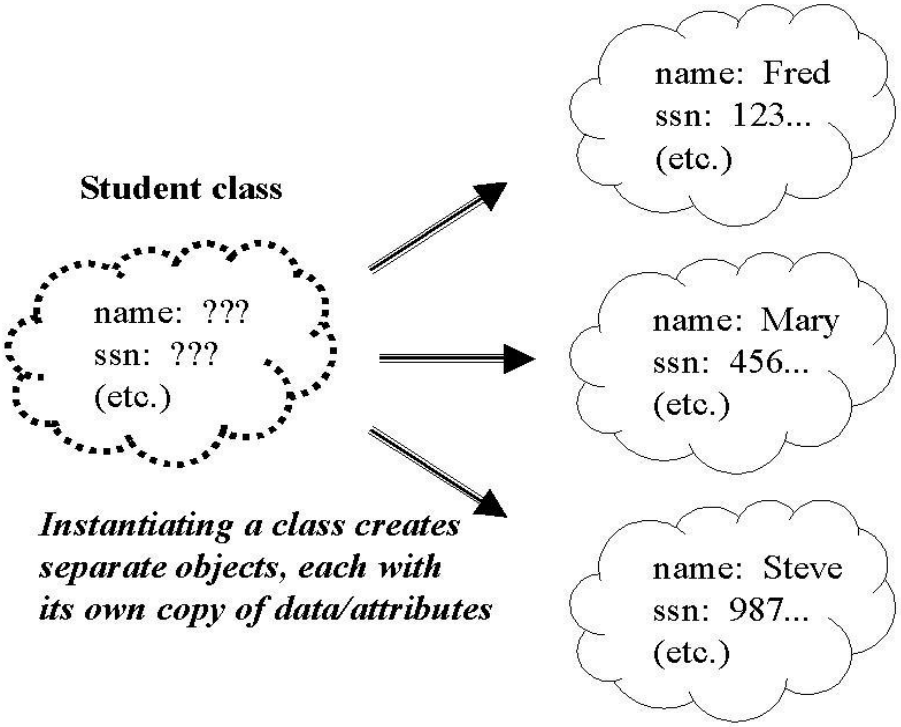
- Whenever possible/feasible, design the public aspects of your classes:
  - arguments to methods
  - method return types
- Using interface types instead of specific class types to allow for greater flexibility/utility of your methods

6.4 Static Features



Static Attributes

- When we create an object, we are creating an instance of the appropriate class 'template' which gets filled in with attribute values



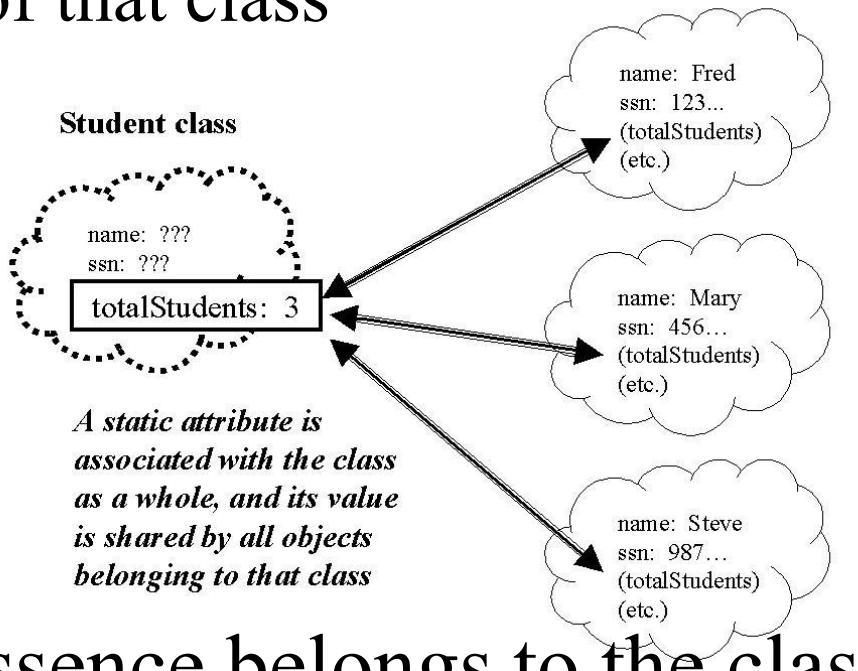
Static Attributes, cont.



- Suppose there were some piece of general information – say, the total student enrollment count at the university – that we wanted *all* Student objects to have shared access to
  - We could implement this as a 'standard' attribute of the Student class -- say, **`int totalStudents`** …
  - … but then we'd have to work hard to make sure that all of the different Student objects' values for **`totalStudents`** were in synch

Static Attributes, cont.

- A **static attribute** is one whose value is shared by all instances of that class



  - It in essence belongs to the class as a whole
  - "Global" to all instances of that class

Static Attributes, cont.

```
class Student {
    private String name;
    private static int totalStudents = 0;

    public Student(String n) {
       setName(n);
       totalStudents++;
    }

    public static int getTotalStudents() {
        return totalStudents;
    }
}
```

Static Attributes, cont.

```
// Client code:
Student s1 = new Student("Fred");
Student s2 = new Student("Mary");
Student s3 = new Student("Steve");


s1.getTotalStudents();
s2.getTotalStudents();
s3.getTotalStudents();
```

Static Attributes, cont.

```
// Client code:
Student s1 = new Student("Fred");
Student s2 = new Student("Mary");
Student s3 = new Student("Steve");

s1.getTotalStudents();
s2.getTotalStudents();
s3.getTotalStudents();

// So will THIS!
Student.getTotalStudents();
```

Accessing Static Attributes



- The latter of these messages illustrates what's known as a **static method**: i.e., one which can be invoked on the class as a whole

```
public class Student {
    private String name;
    private static int totalStudents = 0;
    public static int getTotalStudents() {
          return totalStudents;
     }
    // etc.}
```

Static Methods

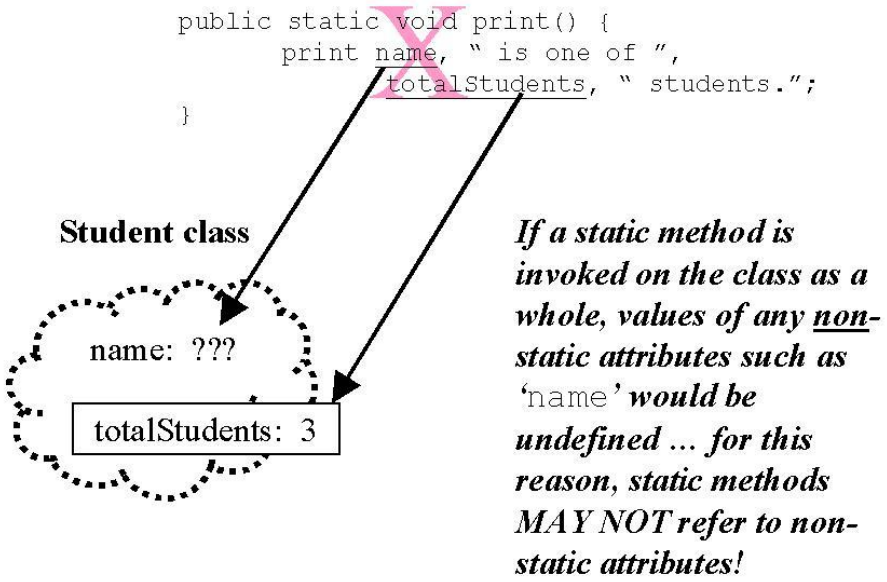


- Important restriction on **static** methods: they may ***only*** access ***static*** attributes:

```
class Student {
     private String name; // NOT static
     private static int totalStudents;
     public static void print() {
    System.out.println(name + " is one of " +
    totalStudents + " students."); // ILLEGAL!
     }
}
```

Static Methods, cont.

- Why is this?
  - Classes are for the most part empty templates
  - If a **static** method is invoked on a class as a whole, and tries to access a non-static attribute, the value of that attribute would be undefined



The Static Nature of main()



- Why must the **main()** method of an application be declared to be static?

```
public static void main(String[] args) { … }
```

- To answer this question, let's review what happens when we first launch an application

```
java MyApp
```

  - the JVM is looking for a class file with the specified name, loading it into its memory, and looking for/running the main() method

The Nature of main(), cont.



```
java MyApp
```

- Step 1: the JVM looks for a class file with the specified name
- Step 2: if found, it is loaded into the JVM's memory
- Step 3: the JVM inspects the class for the existence of a main() method with the appropriate signature
- Step 4: if found, the main() method is executed

-- but, we haven't created have any objects yet, so we must look to the class to run this method => it must be a static method!

Utility Classes



- Some classes consist predominantly/wholly of static methods (and sometimes public static **final** (constant) attributes), and are created as a convenient way of bundling utility functions
  - Example: java.lang.Math
  - Defines static methods for performing basic numeric operations along with elementary exponential, logarithm, square root, and trigonometric functions
  - We never instantiate a Math object -- instead, we use the methods statically on the Math class as a whole, as a sort of math "library"

Math Class Example

```
public class MathExample {
    public static void main(String[] args) {
        // We never instantiate the Math class!
        double d = -3.4;
        double dabs = Math.abs(d);  // absolute value
        double radius = 3.7;
        // A public static (constant) ATTRIBUTE!
        double circumference = 2.0 * Math.PI * radius;
        double area = Math.PI * Math.pow(radius, 2.0);
        double e = 7.92;
        // Compute the square root of d squared plus e
      squared.
        double answer = Math.sqrt(Math.pow(d, 2.0) +
                                  Math.pow(e, 2.0)); //
      nested
        // etc.
    }
```

Alternative Terminology



- To differentiate between ‘normal’ (non-static) attributes and static attributes:
  - ‘**instance variable**’ is often used for the former: namely, a ‘variable’ (attribute) that has value or meaning for an ‘instance’, or object
  - ‘**class variable**’, then, is often used as a synonym for ‘static attribute’

6.5 Design Pattern

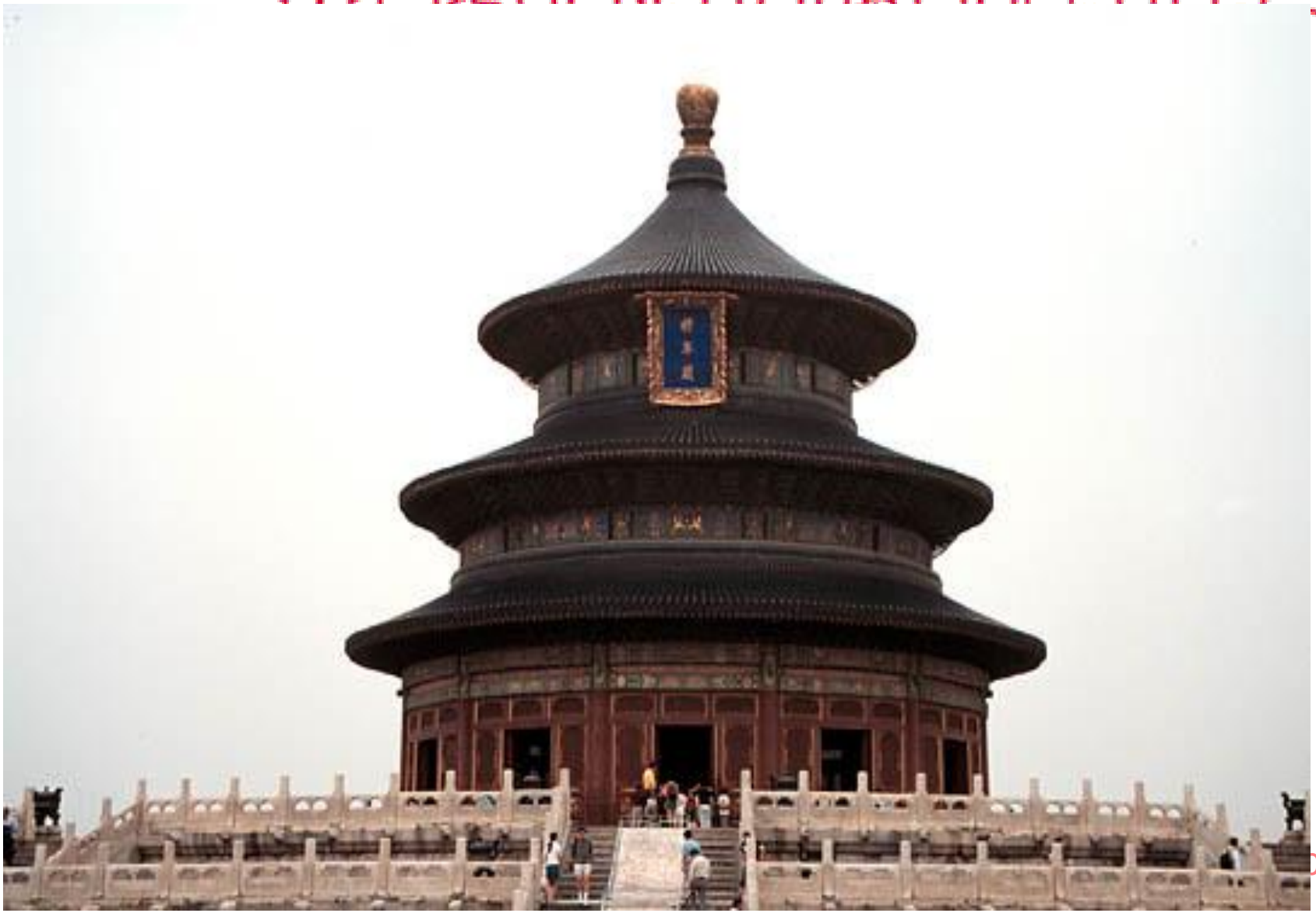


History of Pattern

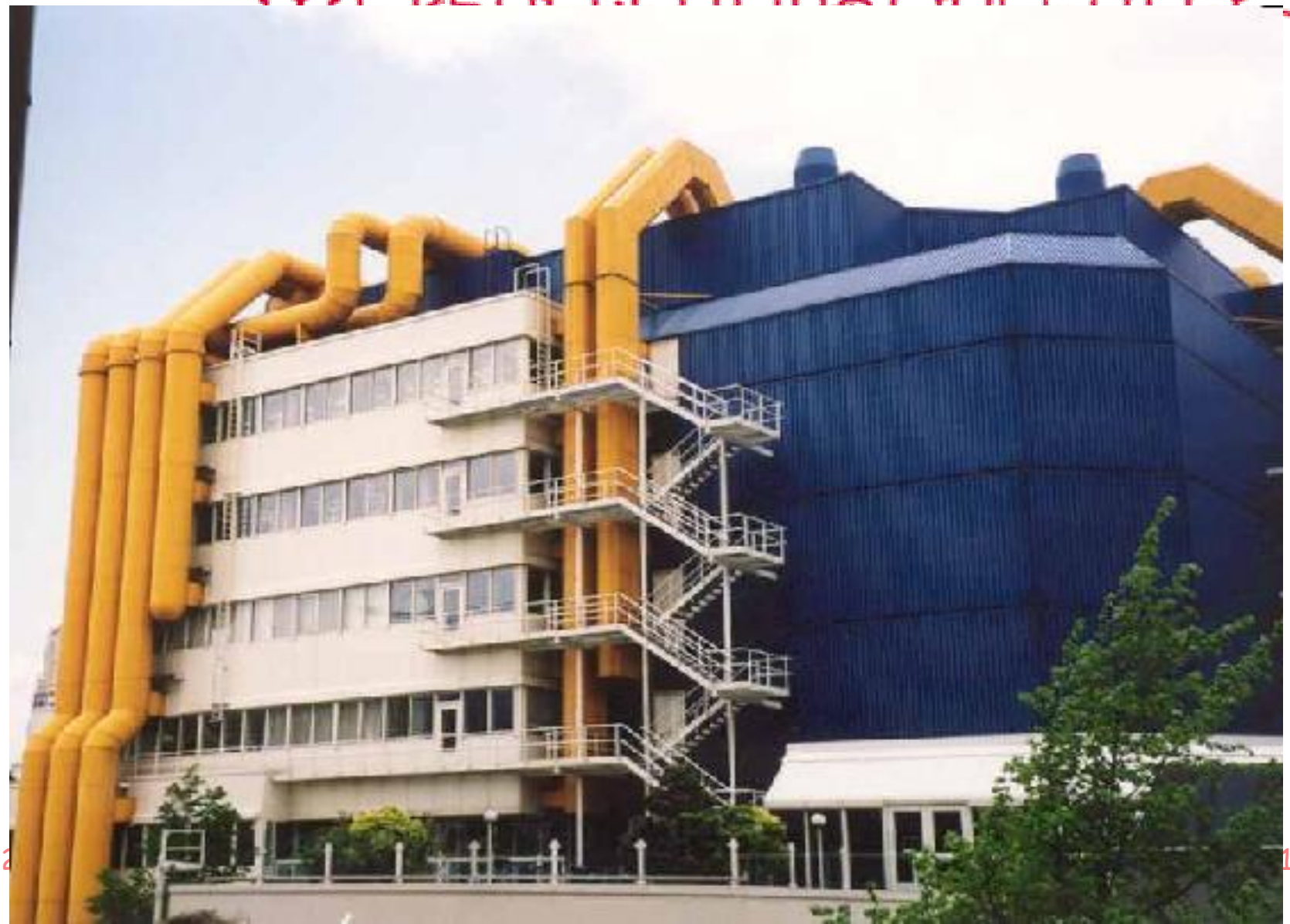


- Christopher Alexander, in the 1970's, wrote books on architectural patterns. He thought that modern architecture was awful and that it ignored expertise that had existed for a long, long time.

Good Design during 1400-1500



The Worst Design in 1900



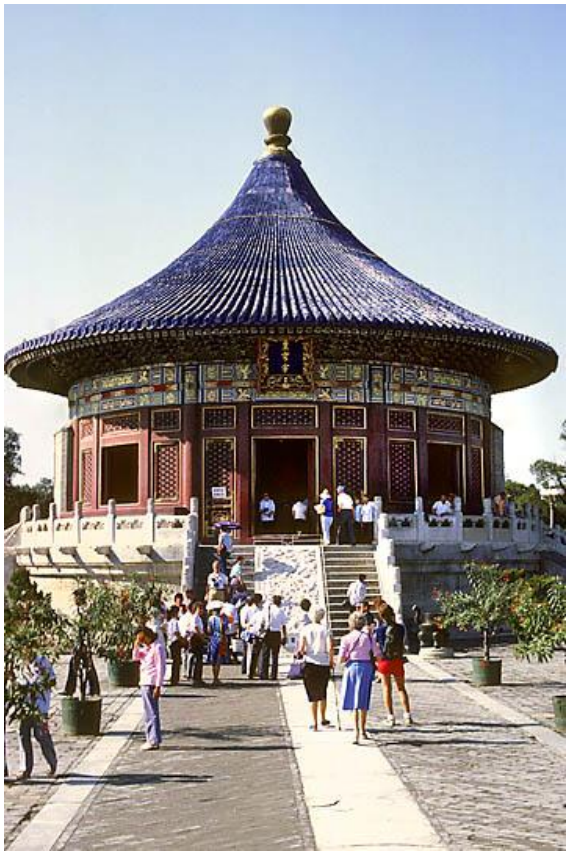
Alexander's Patterns



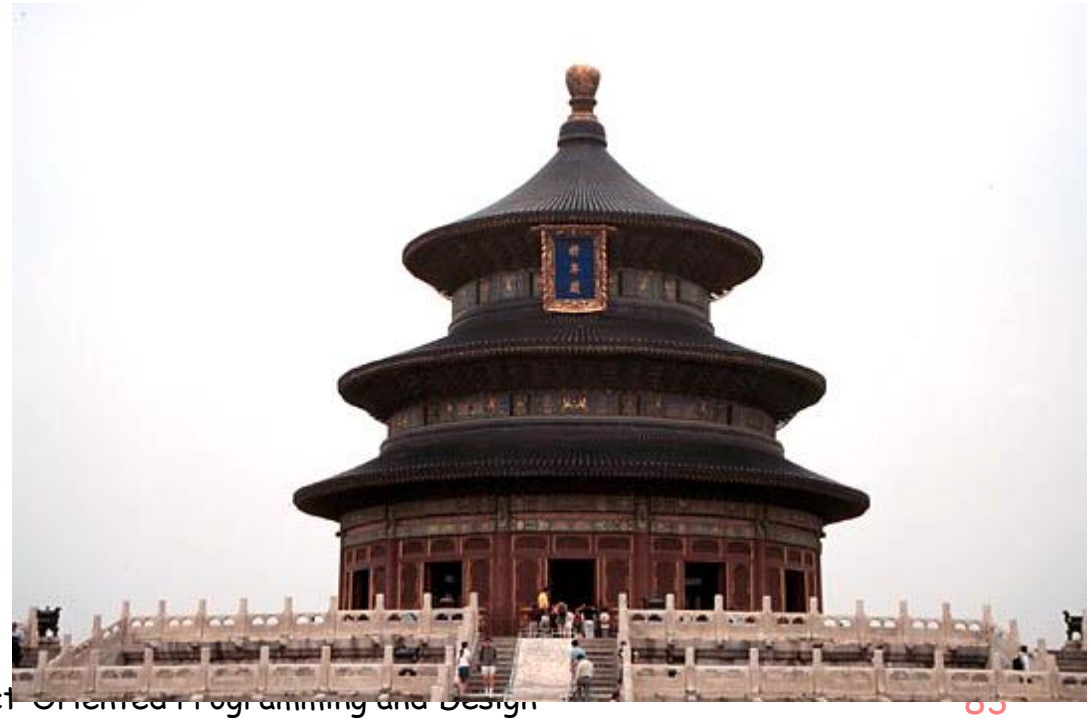




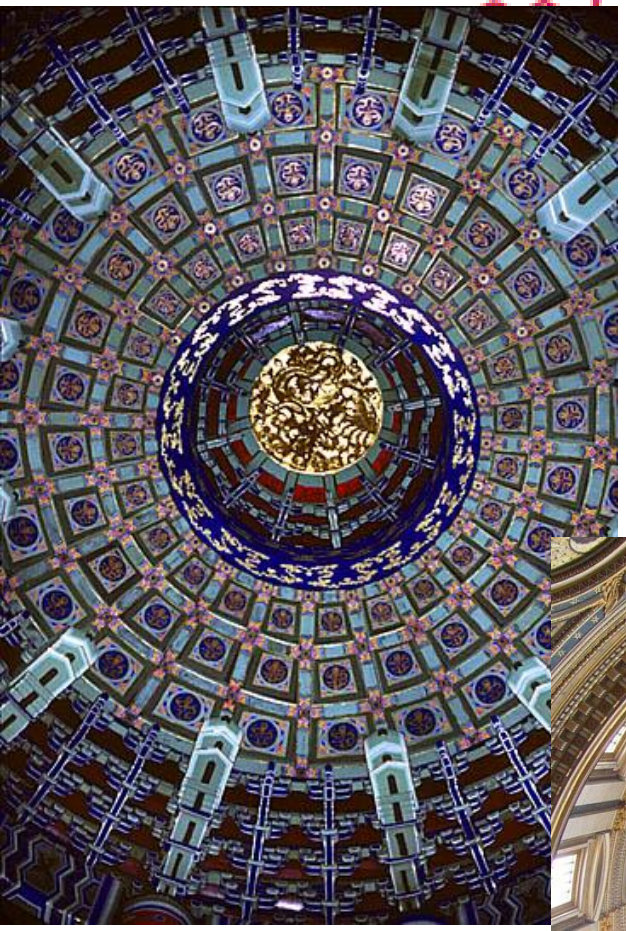
Alexander's Patterns







Alexander's Patterns







Design Pattern



- *Design patterns* describe practical solutions to common design problems that occur repeatedly in software development. A design pattern description consists of:
  - A name that identifies the pattern
  - A description of the problem that the pattern addresses
  - A description of the solution that includes the class structures that solve the problem
  - A discussion of the consequences of using the pattern
- Design patterns document good design solutions that can be used for similar problems and form a shared vocabulary for problem-solving discussions .

Design Patterns



- *Design Patterns: Elements of Reusable Object-Oriented Software*;
- Authors: Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides;

Creational Patterns:	**Abstract Factory, Builder, Factory Method, Prototype , Singleton**
Structural Patterns:	**Adapter, Bridge, Composite, Decorator , Façade, Flyweight, Proxy**
Behavioral Patterns:	**Chain of Responsibility, Command, Iterator, Mediator, Memento, Observer, State, Strategy, Visitor, Template Method, Interpreter**

6.6 Singleton Pattern



Intent



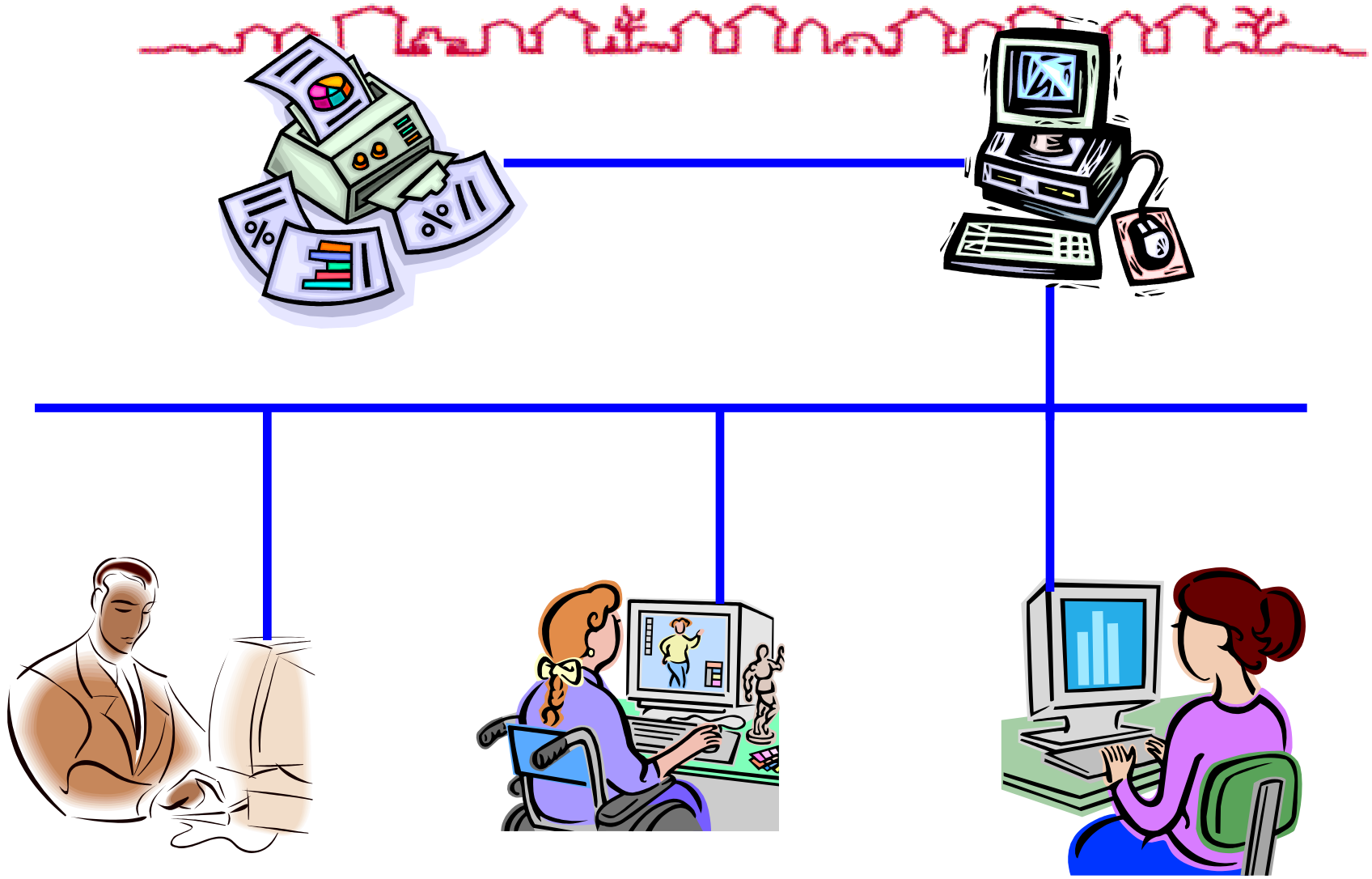
- The intent of the Singleton pattern is to ensure that a class has only one instance and to provide a global point of access to it.

Motivation

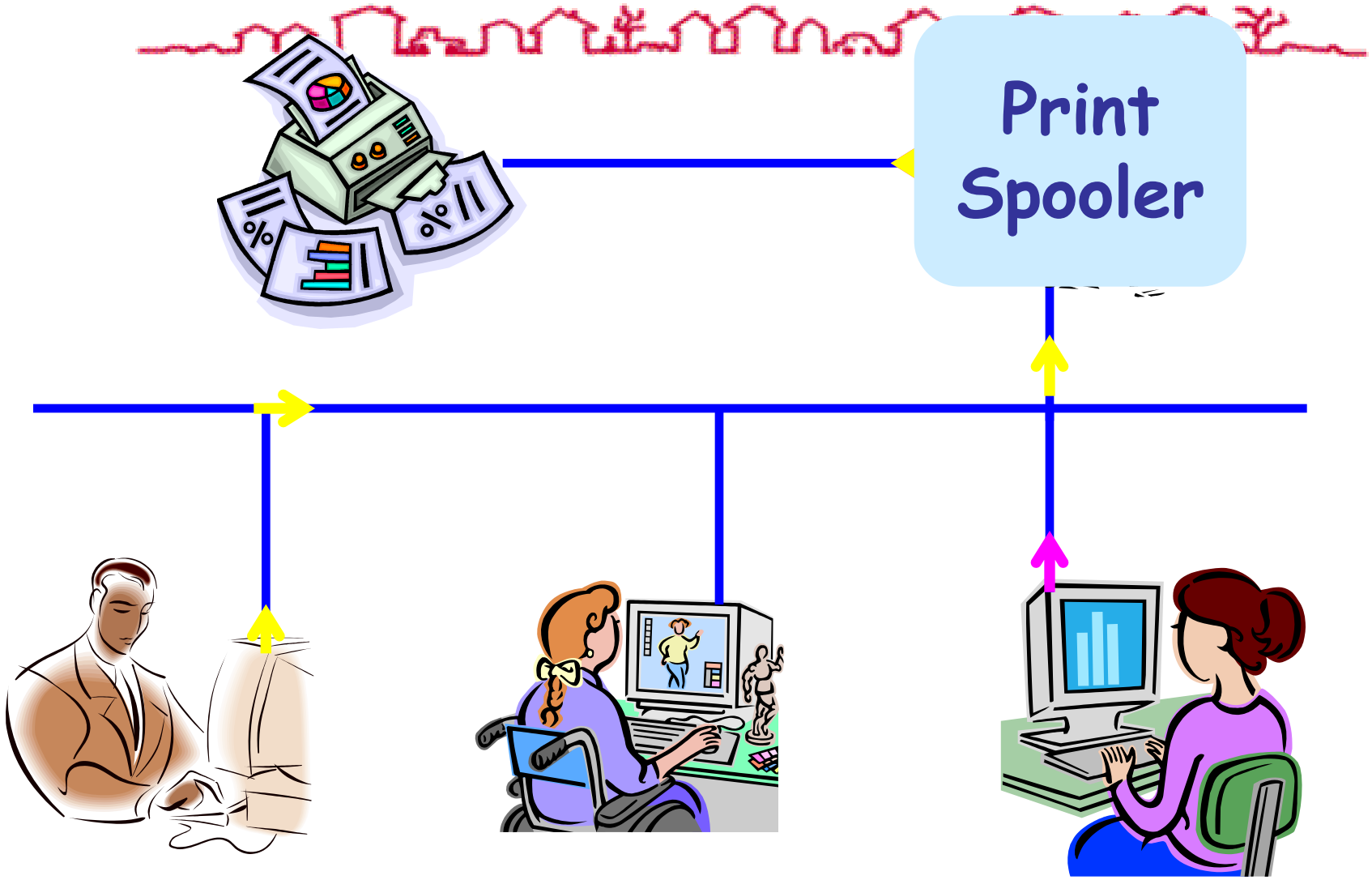


- It's important for some classes to have exactly one instance.
  - Many printers, but only one print spooler
  - One file system
  - One window manager

Example



Example



```
public class PrintSpooler  {
  private String name;
  private String address;
  private Printer printer;

  public PrintSpooler() {
      name = “John";
      address = “192.168.103.11";
      printer = new Printer();
  }

  public void print(String out) {
      printer.print(out);
  }
}
```

```
public class User  {
  private String name;
  private String address;

  public User(String initialName,
              String initialAddress) {
     name = initialName;
     address = initialAddress;
  }

  public void print(PrintSpooler spooler,
                    String content) {
     spooler.print(content);
  }
}
```

```
public class PrintSpoolerDemo  {
  public static void main(String[]  args)  {

    PrintSpooler printSpooler =
                    new PrintSpooler();
    User alice =
         new User("Alice", "192.168.103.12");
    User ann =
         new User("Ann", "192.168.103.13");
    User mike =
         new User("Mike", "192.168.103.14");

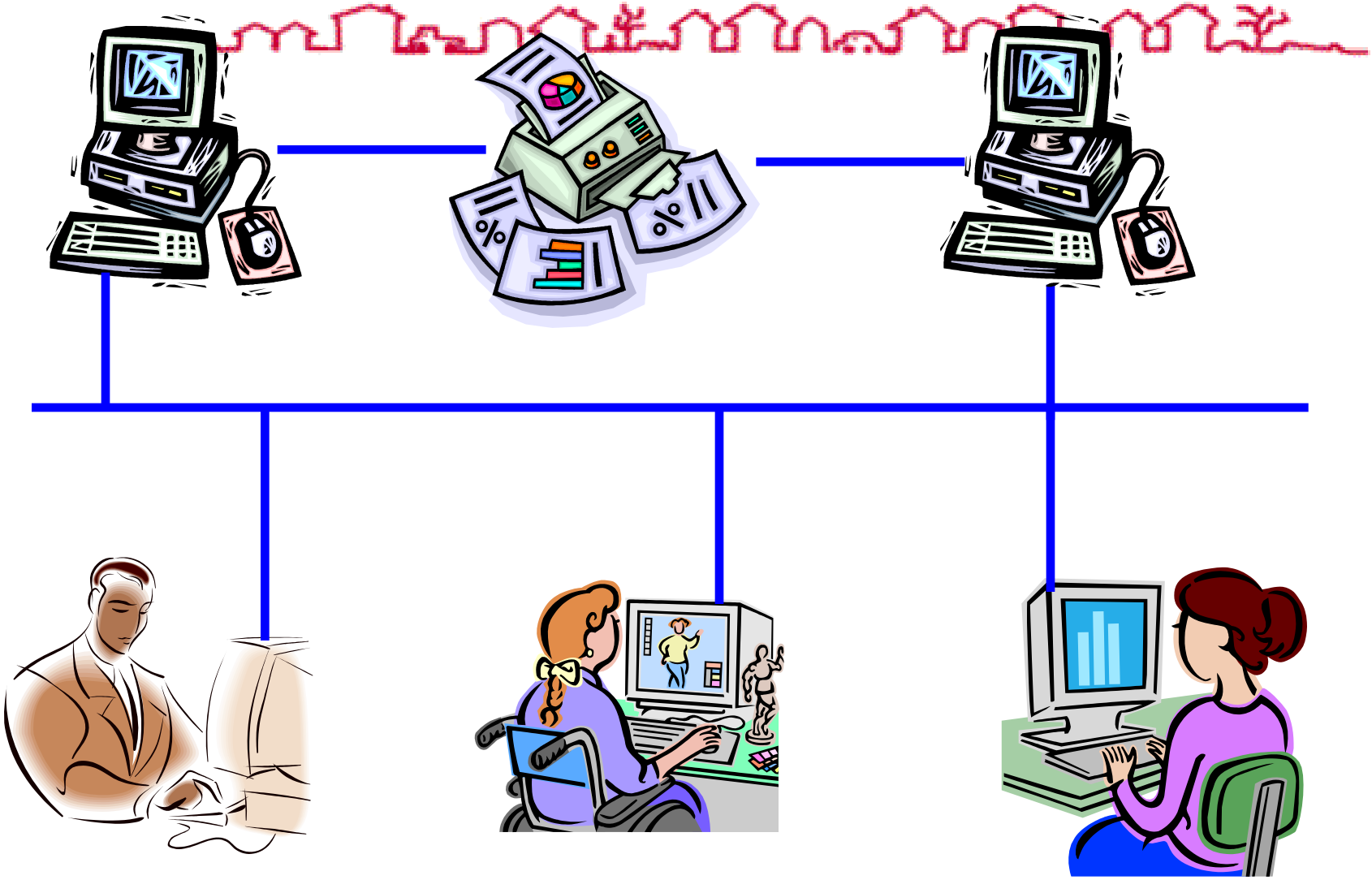
    alice.print(printSpooler, "I am Alice.");
    ann.print(printSpooler, "I am Ann.");
    mike.print(printSpooler, "I am Mike."); }}
```

```
public class PrintSpoolerDemo  {
   public static void main(String[]  args)  {
        PrintSpooler printSpooler =
                                new PrintSpooler();
        User alice =
               new User("Alice", "192.168.103.12");
        User ann =
               new User("Ann", "192.168.103.13");
        User mike =
               new User("Mike", "192.168.103.14");

        PrintSpooler anotherPrintSpooler =
                            new PrintSpooler();

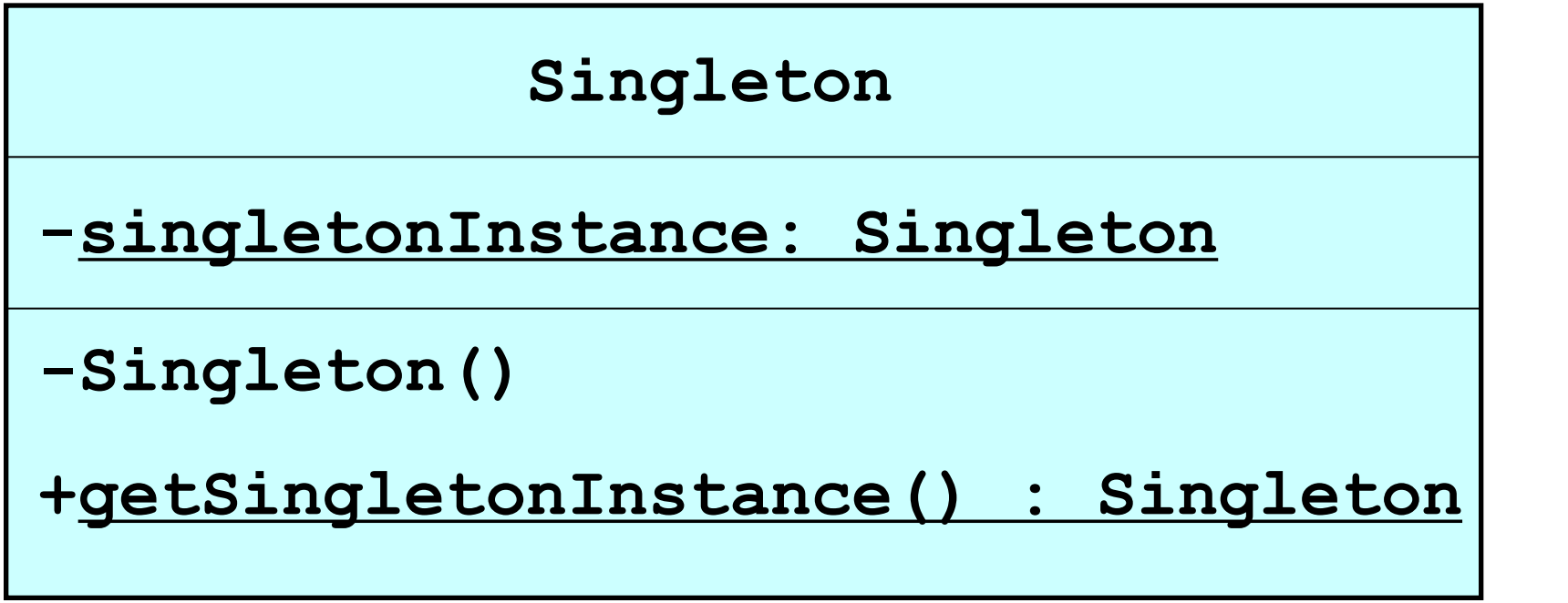
        alice.print(anotherPrintSpooler, "I am Alice.");
        ann.print(printSpooler, "I am Ann.");
        mike.print(anotherPrintSpooler, "I am Mike.");
   }
}
```

Example



Singleton Structure

- Structure:



- Notice that the static members of the class are underlined.

In this pattern

- The constructor is defined as **`private`** so that other classes cannot create instances.
- The **`static`** attribute instance contains the single instance of the class.
- The **`static`** method `getSingletonInstance` returns the single instance of the class. The first time this method is called, it creates the single instance.

Example----PrintSpooler



PrintSpooler
-printSpoolerInstance: PrintSpooler
-PrintSpooler() +getPrintSpoolerInstance() : PrintSpooler

```
public class PrintSpooler  {
   private String name;
   private String address;
   private Printer printer;
   static private PrintSpooler printSpoolerInstance = null;
   private PrintSpooler() {
         name = "John";
         address = "192.168.103.11";
         printer = new Printer();
   }
   static public PrintSpooler getPrintSpoolerInstance(){
         if(printSpoolerInstance == null){
               printSpoolerInstance = new PrintSpooler();
         }
         return printSpoolerInstance;
   }
   public void print(String out) {
       printer.print(out);
   }
}
```

```
public class PrintSpoolerDemo  {
   public static void main(String[]  args)  {

      PrintSpooler printSpooler =
            PrintSpooler.getPrintSpoolerInstance();

      User alice =
            new User("Alice", "192.168.103.12");
      User ann =
            new User("Ann", "192.168.103.13");
      User mike =
            new User("Mike", "192.168.103.14");

      PrintSpooler anotherPrintSpooler =
            PrintSpooler.getPrintSpoolerInstance();

      alice.print(anotherPrintSpooler, "I am Alice.");
      ann.print(printSpooler, "I am Ann.");
      mike.print(anotherPrintSpooler, "I am Mike.");
   }
}
```

Program run

- In main method

```
PrintSpooler printSpooler =
        PrintSpooler.getPrintSpoolerInstance();
```



Program run

- In getPrintSpoolerInstance method.

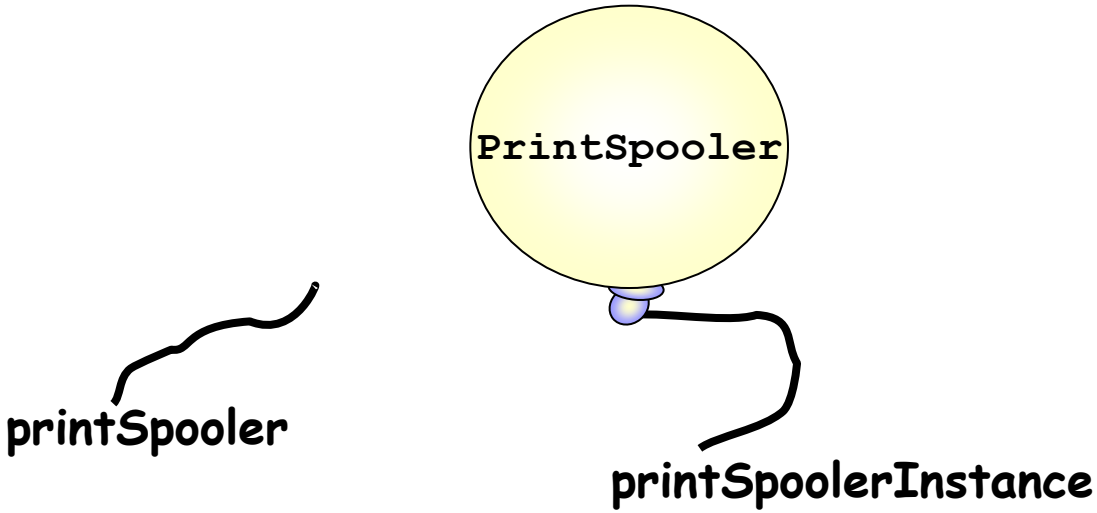
```
if(printSpoolerInstance == null){
      printSpoolerInstance = new PrintSpooler();
}
return printSpoolerInstance;
```



Program run

- In getPrintSpoolerInstance method.

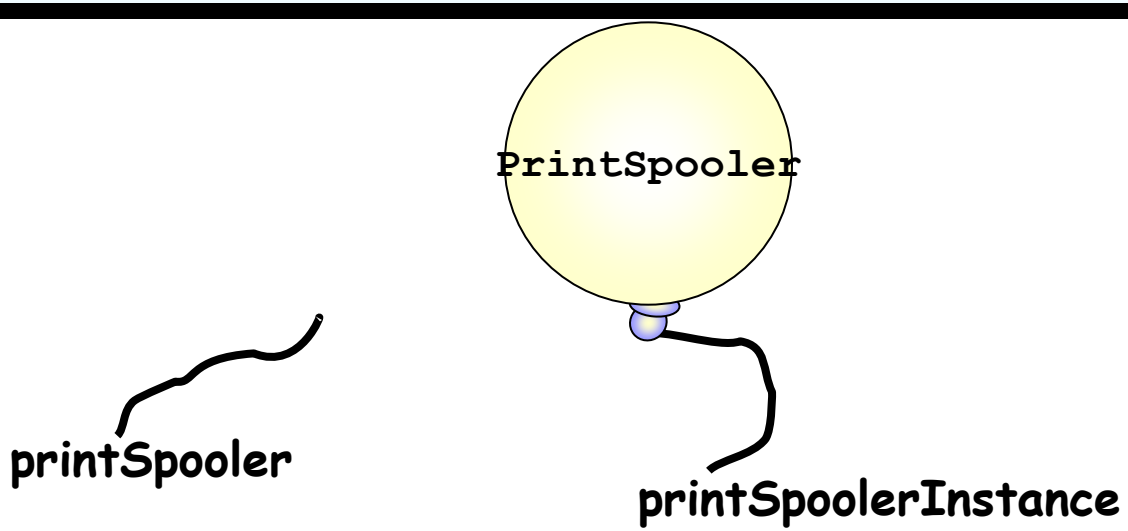
```
if(printSpoolerInstance == null){
        printSpoolerInstance = new PrintSpooler();
}
return printSpoolerInstance;
```



Program run

- In getPrintSpoolerInstance method.

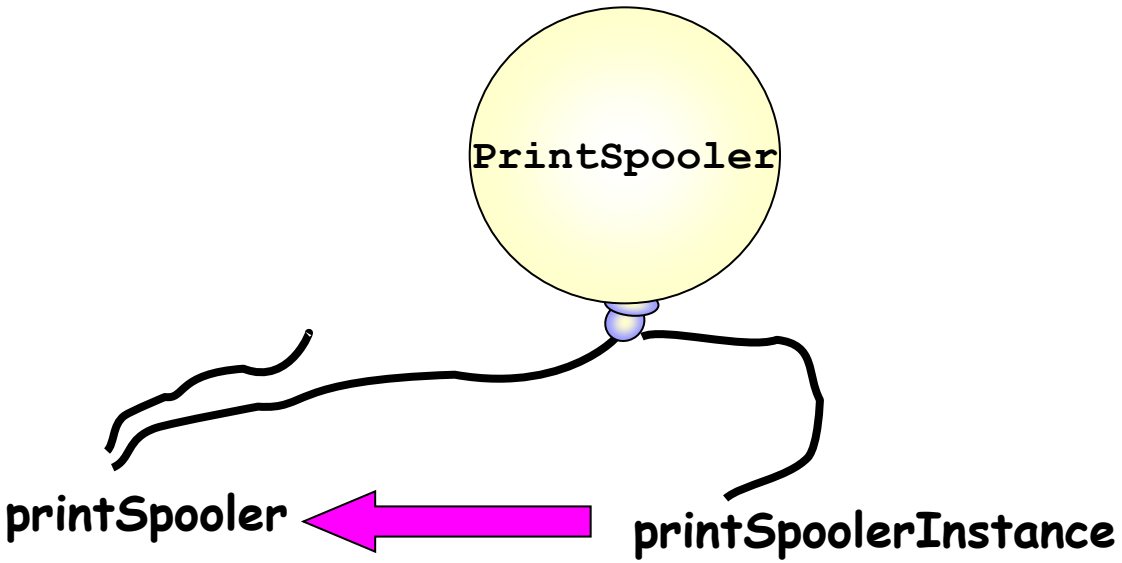
```
if(printSpoolerInstance == null){
      printSpoolerInstance = new PrintSpooler();
}
return printSpoolerInstance;
```



Program run

- Back main method.

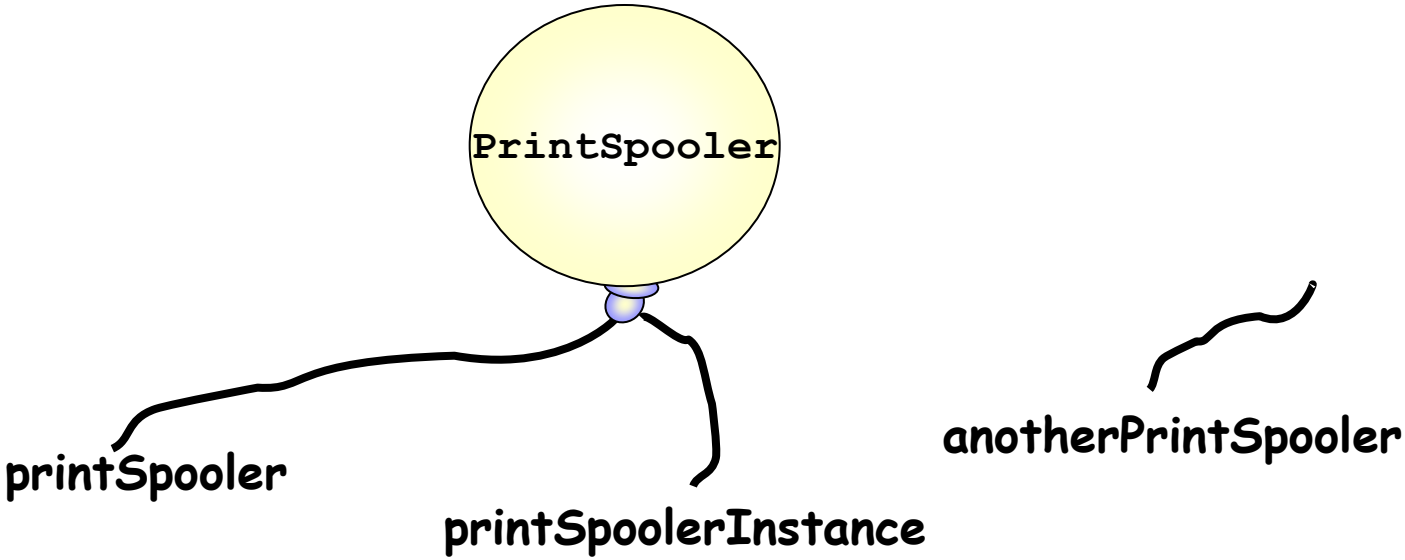
```
PrintSpooler printSpooler =
  PrintSpooler.getPrintSpoolerInstance();
```



Program run

- In main method.

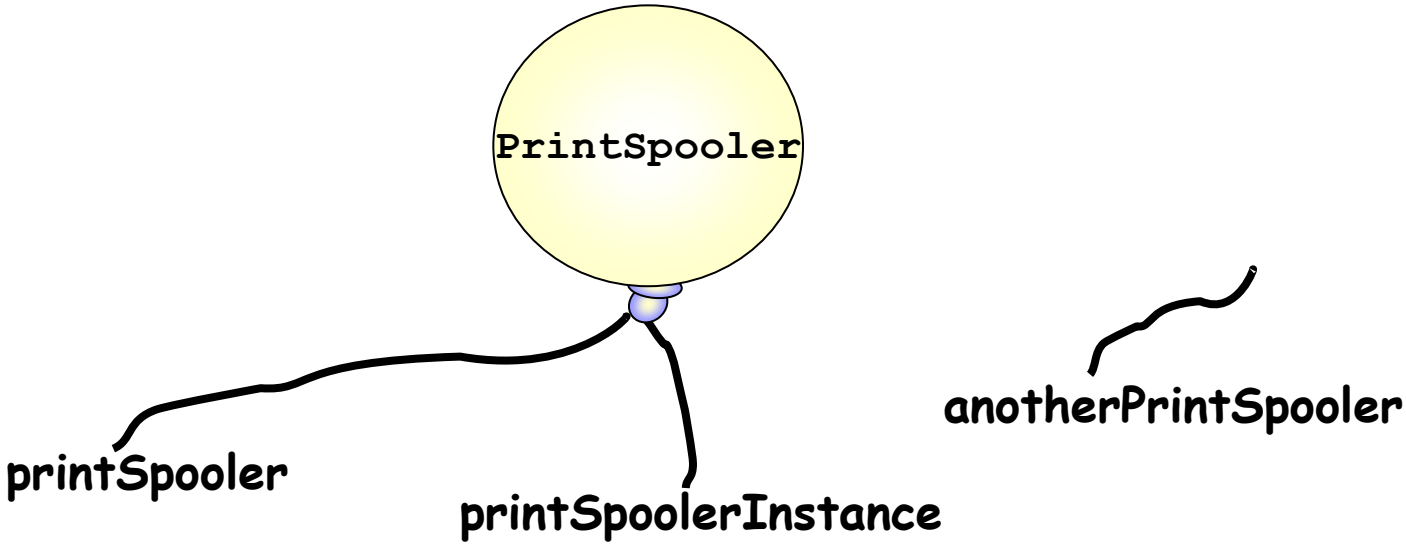
```
PrintSpooler anotherPrintSpooler =
        PrintSpooler.getPrintSpoolerInstance();
```



Program run

- In getPrintSpoolerInstance method.

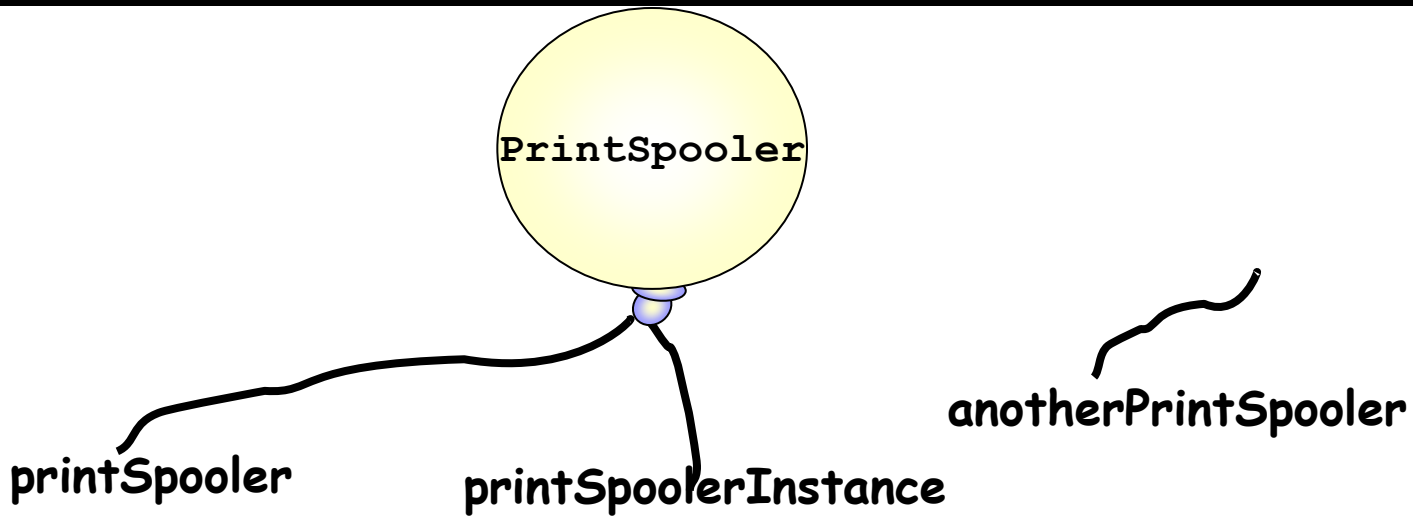
```
if(printSpoolerInstance == null){
        printSpoolerInstance = new PrintSpooler();
}
return printSpoolerInstance;
```



Program run

- In getPrintSpoolerInstance method.

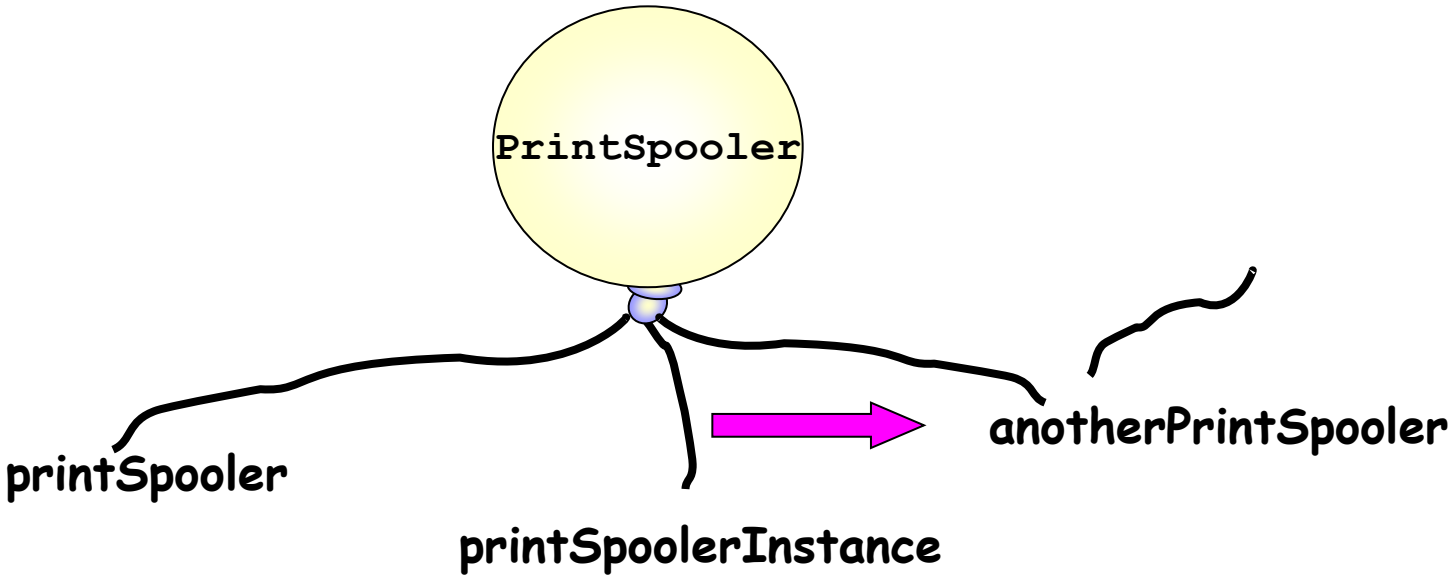
```
if(printSpoolerInstance == null){
      printSpoolerInstance = new PrintSpooler();
}
return printSpoolerInstance;
```



Program run

- Back main method.

```
PrintSpooler anotherPrintSpooler =
        PrintSpooler.getPrintSpoolerInstance();
```



```
public class PrintSpoolerDemo  {
   public static void main(String[]  args)  {

      PrintSpooler printSpooler =
            PrintSpooler.getPrintSpoolerInstance();


      User alice =
            new User("Alice", "192.168.103.12");
      User ann =
            new User("Ann", "192.168.103.13");
      User mike =
            new User("Mike", "192.168.103.14");

      PrintSpooler anotherPrintSpooler =
            PrintSpooler.getPrintSpoolerInstance();

      alice.print(anotherPrintSpooler, "I am Alice.");
      ann.print(printSpooler, "I am Ann.");
      mike.print(anotherPrintSpooler, "I am Mike.");

   }
}
```

Singleton Pattern



- The singleton pattern ensures that only one instance of a class is created and provides a method to access that one instance.
- Every object that uses an instance of a singleton class uses the same instance.

Key of Singleton Pattern



- Constructors must be private. Notice that all constructors must be private.
- Must provide a public static method to issue and keep track of instances.

Benefits



- A singleton class can control how and when client code accesses the single instance.
- A singleton class can be easily modified if requirements change and the application needs to limit the number of instances to a number other than one.

6.7 Strategy Pattern

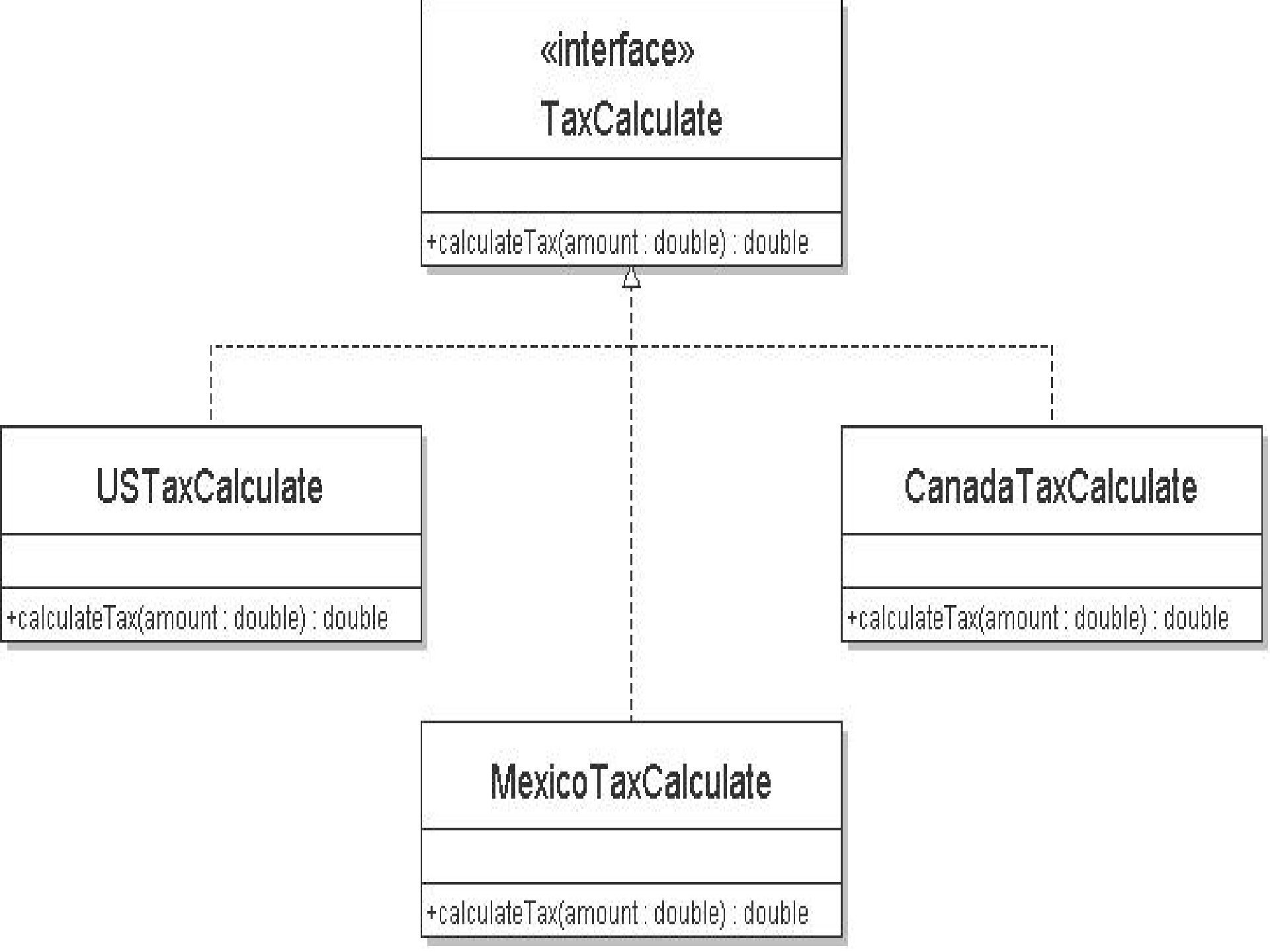


For example: sales-tax calculation

- Consider an e-commerce system that supports sales in the United States, Mexico, and Canada. In the near future, the customer base may expand to include countries in South America. The system must know how to calculate the sales tax for customers in three countries, each of which has its own rules for calculating sales tax. The design of the code for sales-tax calculation should be flexible so it will be easy to add other countries. One solution is a monolithic method called calculateTax that calculates the sales taxes for every country.

sales-tax calculation (cont.)

```
public class Order  {
private double calculateTax(String country, double
   amount)  {
        double tax
        if (country.equals("United States")) {
            tax = ... ;
        } else if (country.equals("Canada")) {
            tax = ... ;
        } else if (country.equals("Mexico")) {
            tax = ... ;
        }
        ...
        return tax;
    }
}
```


«interface»
TaxCalculate
+calculateTax(amount : double) : double
USTaxCalculate
+calculateTax(amount : double) : double
CanadaTaxCalculate
+calculateTax(amount : double) : double
MexicoTaxCalculate
+calculateTax(amount : double) : double

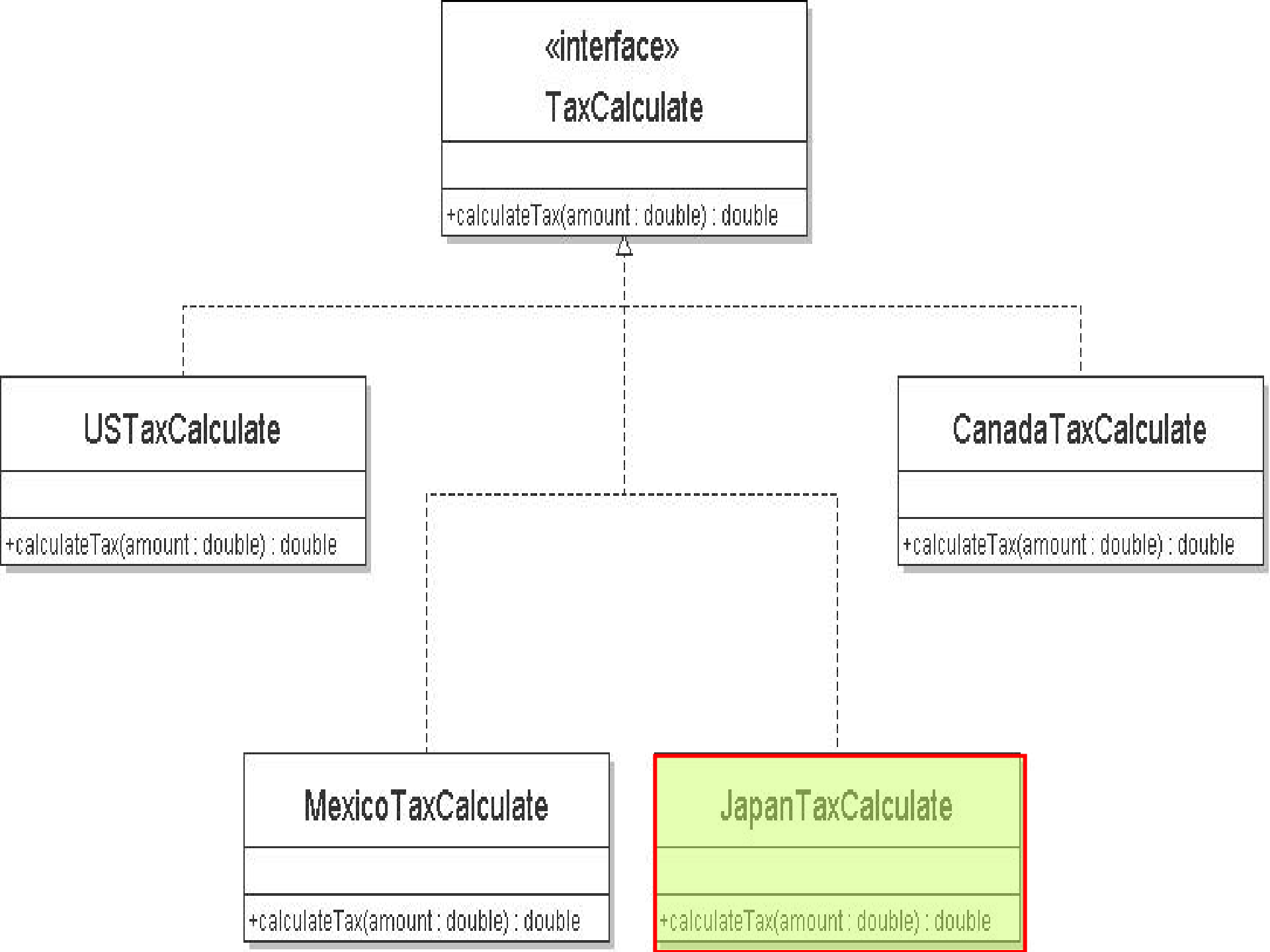
```
public class TestTaxCaculate {

  public static void main(String[] args){
     TaxCalculate taxCalculate;

     taxCalculate = new USTaxCalculate();
     taxCalculate.calculateTax(1000000);

     taxCalculate = new MexicoTaxCalculate();
     taxCalculate.calculateTax(1000000);

     taxculate = new CanadaTaxCalculate();
     taxCalculate.calculateTax(10000000);
  }
}
```


«interface»
TaxCalculate
+calculateTax(amount : double) : double
USTaxCalculate
+calculateTax(amount : double) : double
CanadaTaxCalculate
+calculateTax(amount : double) : double
MexicoTaxCalculate
+calculateTax(amount : double) : double
JapanTaxCalculate
+calculateTax(amount : double) : double

```
public class TestTaxCaculate {

  public static void main(String[] args){

      TaxCalculate taxCalculate;

      taxCalculate = new USTaxCalculate();
      taxCalculate.calculateTax(1000000);

      taxCalculate = new MexicoTaxCalculate();
      taxCalculate.calculateTax(1000000);

      taxculate = new CanadaTaxCalculate();
      taxCalculate.calculateTax(10000000);

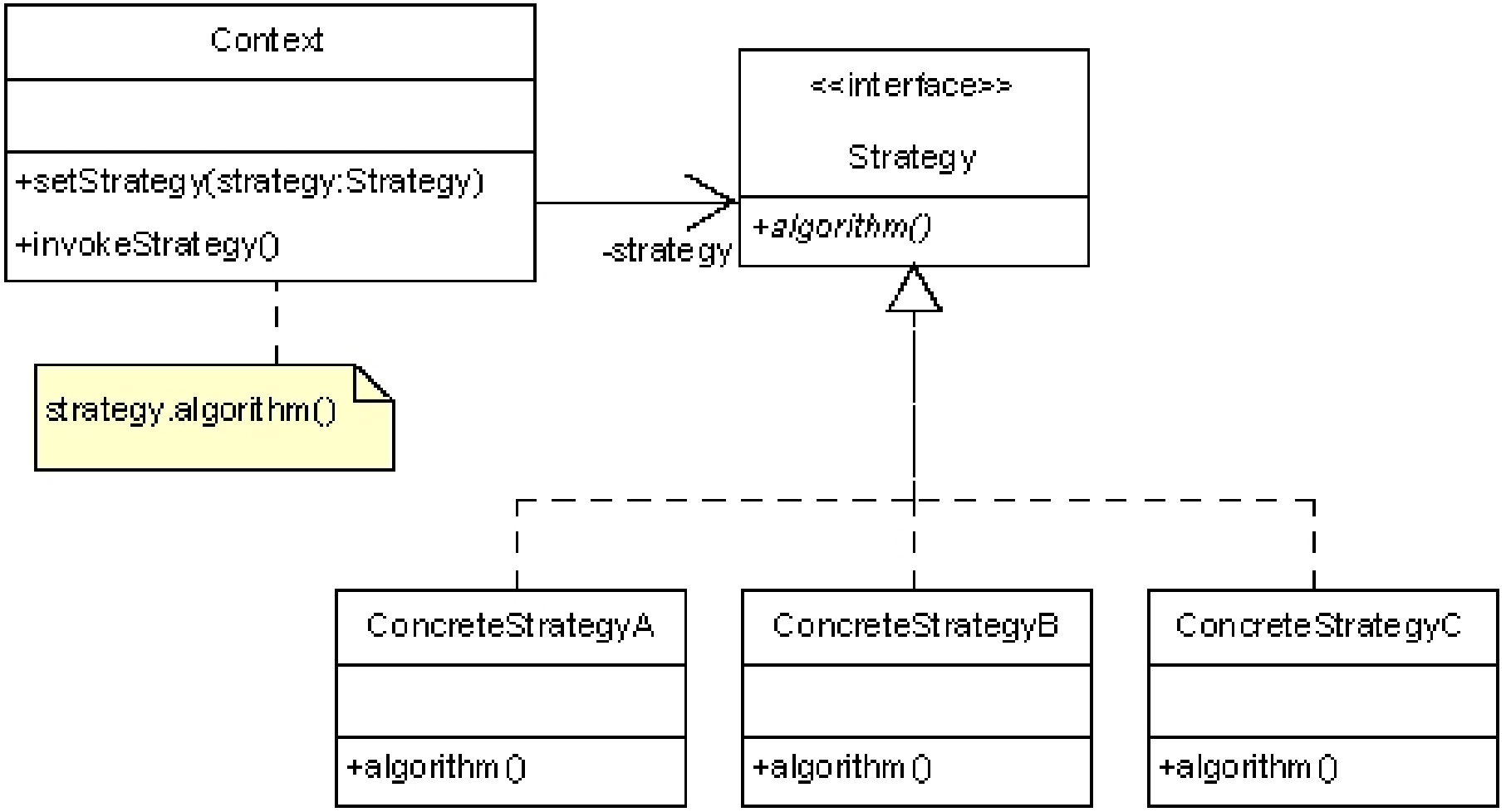
      taxculate = new JapanTaxCalculate();
      taxCalculate.calculateTax(10000000);
  }
}
```

strategy pattern



- Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from the clients that use it.
  - each algorithm is encapsulated in its own class.
  - An interface that declares a method for executing an algorithm is created, and all the algorithm classes implement the interface .
- Applications where the strategy pattern may prove useful include:
  - An application for compressing files that allows the user to select the type of compression
  - An application where the nature of the data to be sorted determines the sorting algorithm

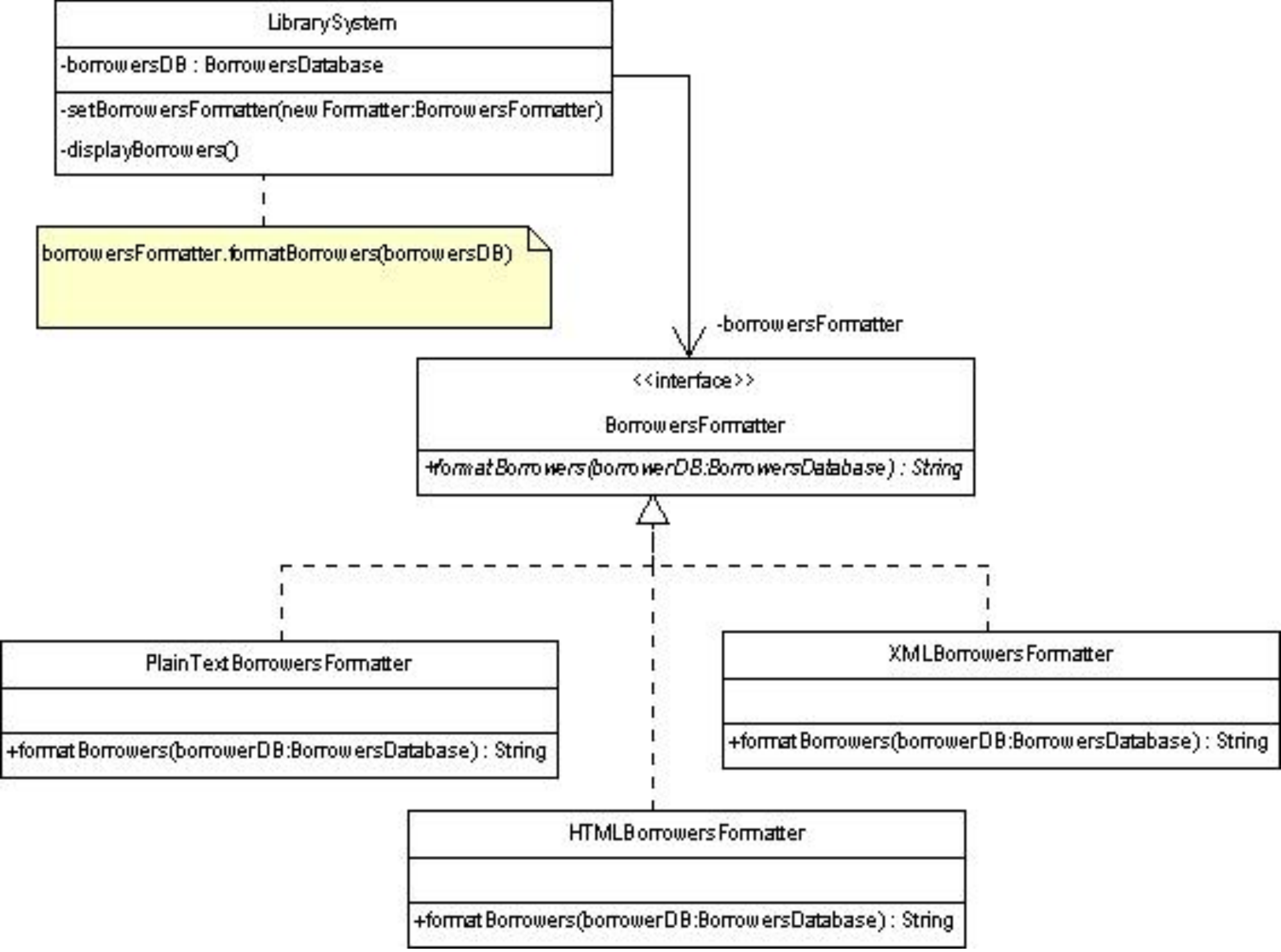
strategy pattern- UML Class Diagram



library system



- This enhancement makes it possible to format the information in the borrower database in one of three ways:
  - plain text,
  - HTML
  - XML


LibrarySystem
-borrowersDB : BorrowersDatabase
-setBorrowersFormatter(new Formatter:BorrowersFormatter)
-displayBorrowers()
borrowersFormatter.formatBorrowers(borrowersDB)
-borrowersFormatter
<<interface>>
BorrowersFormatter
+formatBorrowers(borrowerDB:BorrowersDatabase) : String
PlainTextBorrowersFormatter
+formatBorrowers(borrowerDB:BorrowersDatabase) : String
XMLBorrowersFormatter
+formatBorrowers(borrowerDB:BorrowersDatabase) : String
HTMLBorrowersFormatter
+formatBorrowers(borrowerDB:BorrowersDatabase) : String

Exercises



- Read textbook, chapter 7.
- Give an example to use singleton and strategy pattern.
- To study a new pattern of iterator and give an example to use iterator.